Environment Setup – Anaconda and Jupyter Notebook

Environment Setup – Anaconda and Jupyter Notebook

1. Introduction

This lab introduces students to the foundational tools commonly used in data science and analytics: Anaconda and Jupyter Notebook. Both are essential components of a modern Python-based data science workflow. This session is designed to guide students through the installation and configuration of these tools, as well as provide an overview of their roles and practical applications.

2. Objectives

By the end of this lab, students will be able to:

- Understand what Anaconda and Jupyter Notebook are and why they are important in data science
- Install Anaconda on their system
- Launch and use Jupyter Notebook for writing and executing Python code
- Understand the basic structure and usage of a notebook environment
- Perform basic operations in Python using Jupyter Notebook

3. Overview of Tools

3.1 What is Anaconda?

Anaconda is an open-source distribution of the Python and R programming languages, specifically tailored for scientific computing, data analysis, and machine learning. It simplifies package management and deployment and includes a wide range of pre-installed libraries and tools used in data science.

Key Advantages of Using Anaconda:

Bundles essential data science packages (e.g., NumPy, pandas, matplotlib, scikit-learn)

Includes integrated tools like Jupyter Notebook, Spyder, and Anaconda Navigator

Simplifies package installation and environment management via conda

Cross-platform support (Windows, macOS, Linux)

3.2 What is Jupyter Notebook?

Jupyter Notebook is a browser-based interactive development environment that allows users to write and execute Python code in a readable and modular format. It supports both code execution and rich documentation (including markdown, equations, and visualizations) within the same file, making it especially useful for analysis, reporting, and educational purposes.

Why Use Jupyter Notebook?

- Facilitates step-by-step coding and analysis in segmented cells
- Supports inline visualization of data
- Encourages reproducibility and transparency in analytical work
- Widely adopted in academic and industry settings

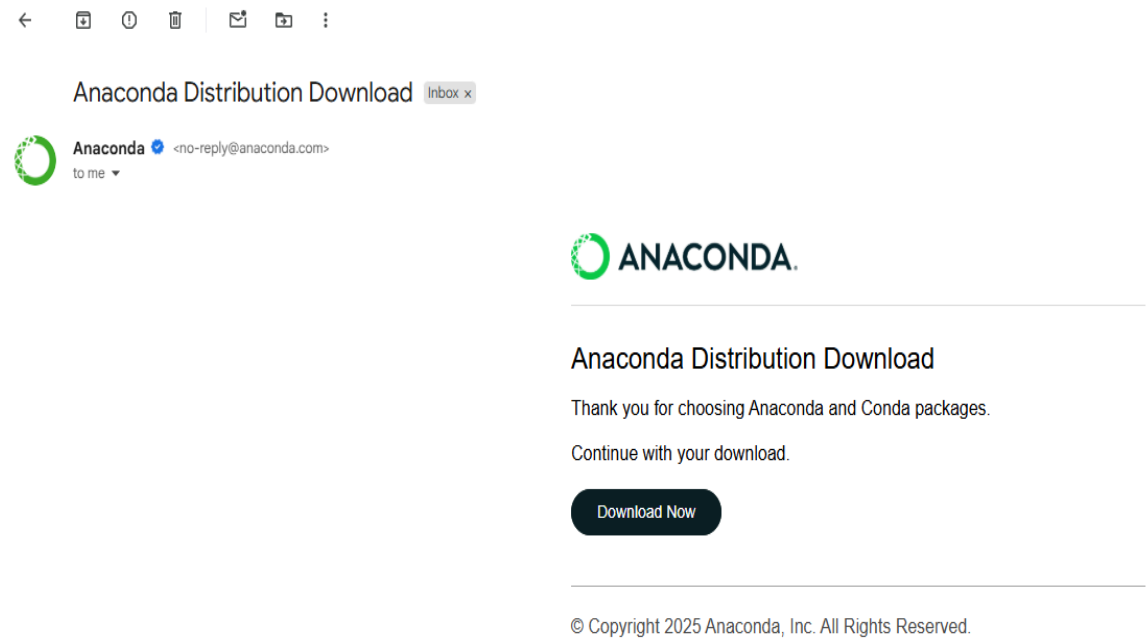
4. Why This Environment is Recommended

The combination of Anaconda and Jupyter Notebook provides a robust, user-friendly platform for both beginners and experienced practitioners in data science. Anaconda manages the software ecosystem efficiently, while Jupyter Notebook offers a flexible interface for exploratory programming, analysis, and documentation. Together, they enable a structured and productive learning experience.

5. Installation guide and usage

Goto the following link and follow the steps below.

https://www.anaconda.com/download



Anaconda Distribution Download Inbox ×

Anaconda <no-reply@anaconda.com>
to me

ANACONDA.

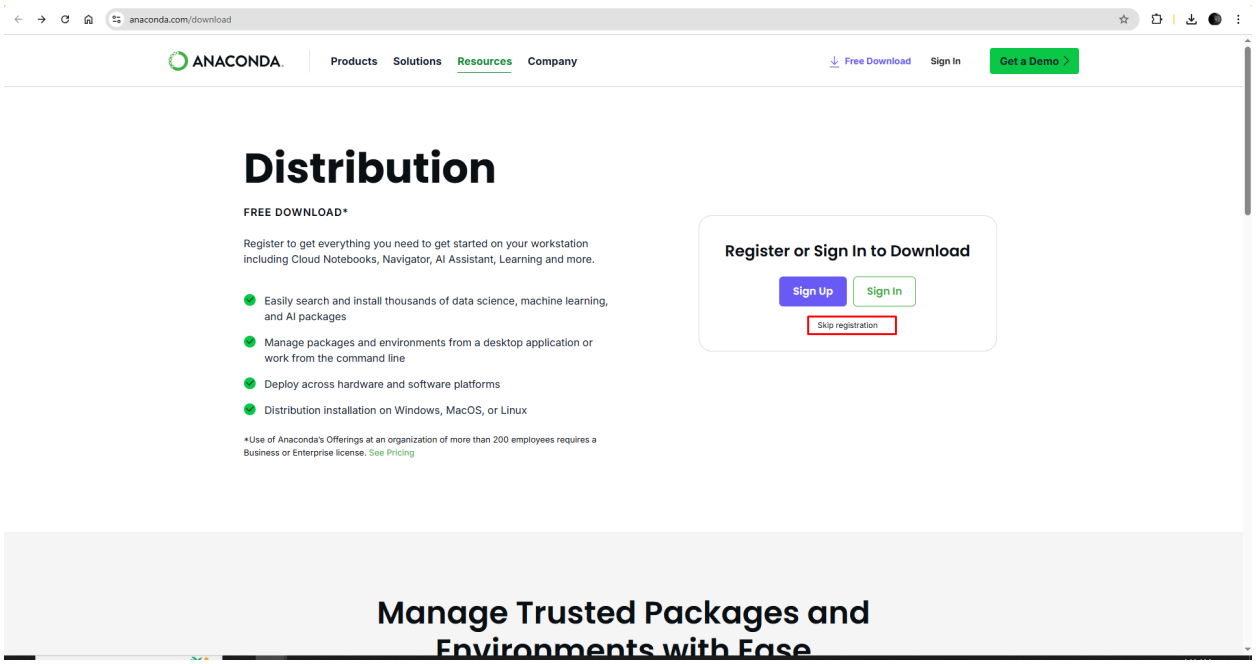
Anaconda Distribution Download

Thank you for choosing Anaconda and Conda packages.

Continue with your download.

Download Now

© Copyright 2025 Anaconda, Inc. All Rights Reserved.



anaconda.com/download

ANACONDA. Products Solutions Resources Company Free Download Sign In Get a Demo

Distribution

FREE DOWNLOAD*

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- Easily search and install thousands of data science, machine learning, and AI packages
- Manage packages and environments from a desktop application or work from the command line
- Deploy across hardware and software platforms
- Distribution installation on Windows, MacOS, or Linux

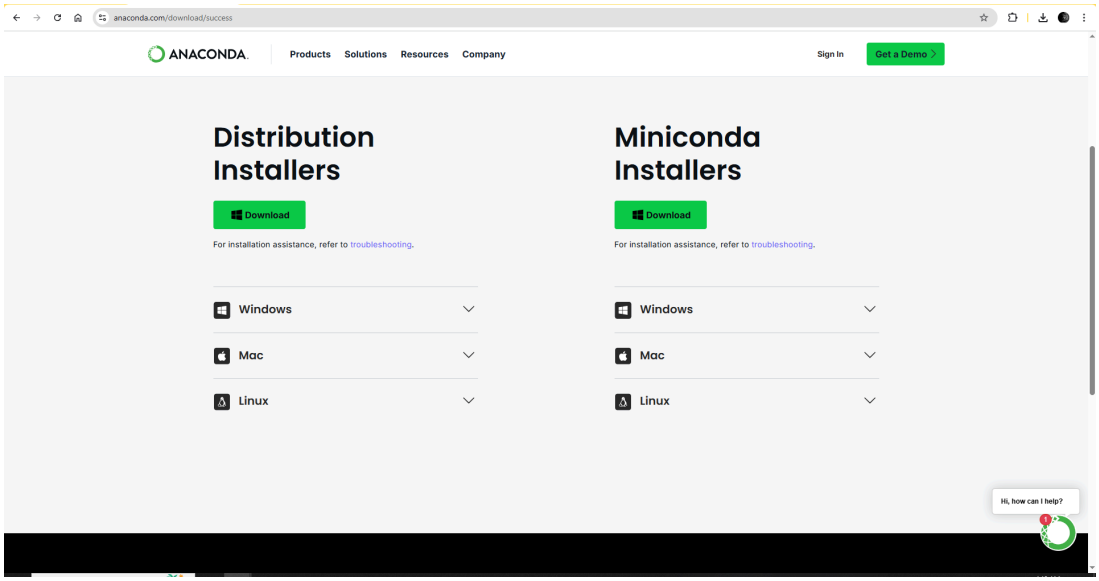
*Use of Anaconda's Offerings at an organization of more than 200 employees requires a Business or Enterprise license. See Pricing

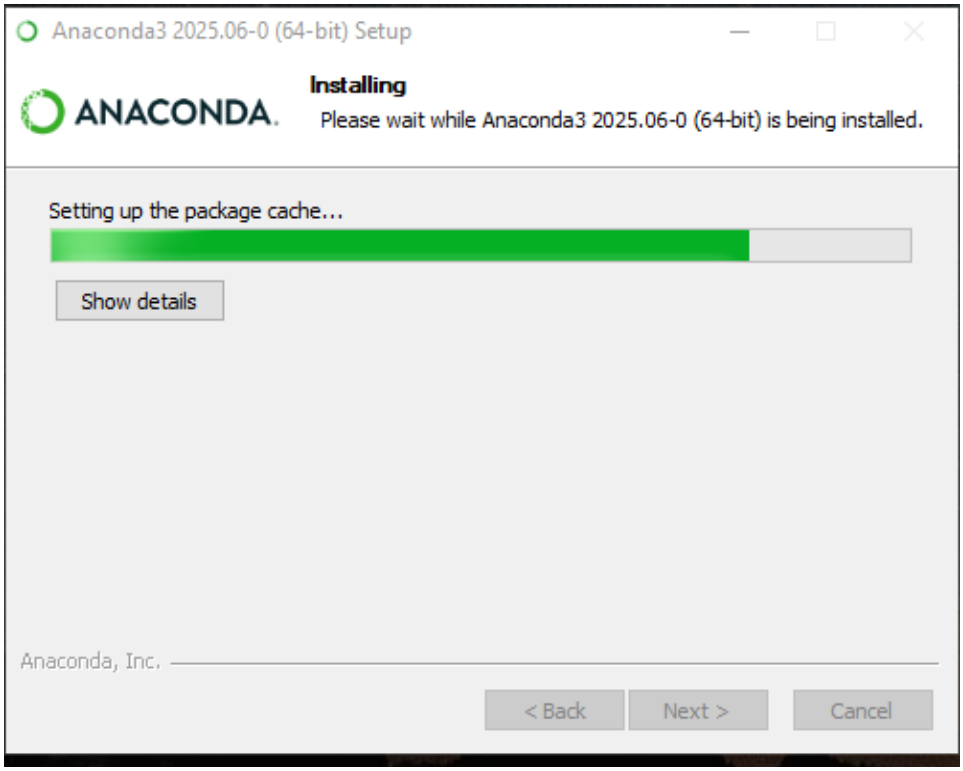
Register or Sign In to Download

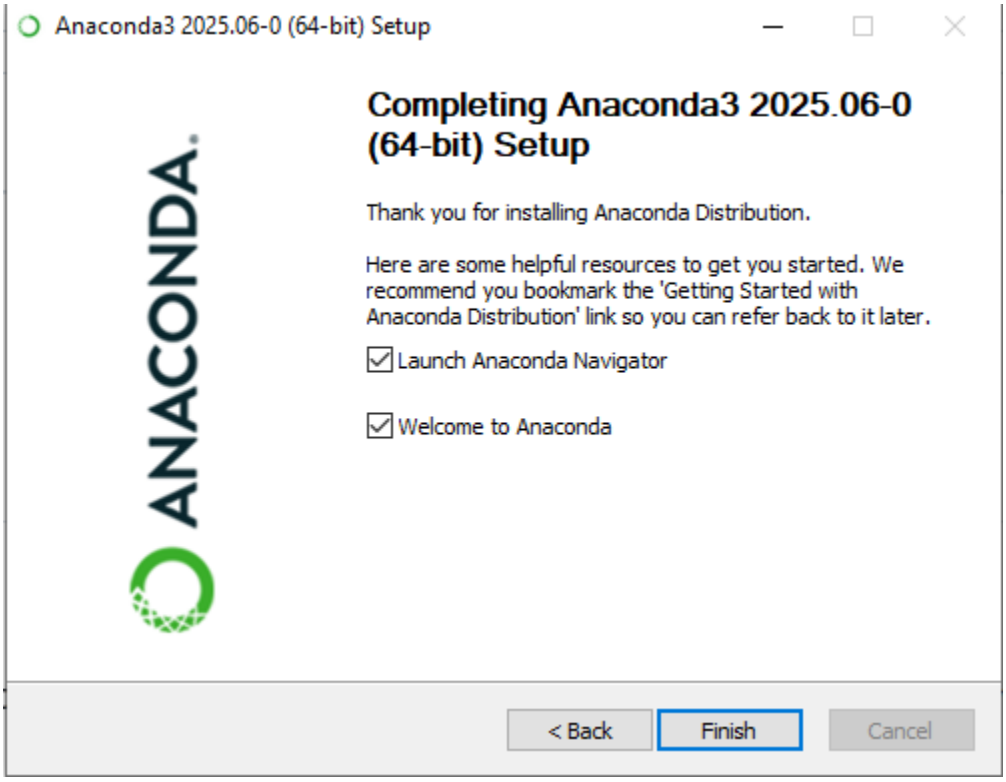
Sign Up Sign In

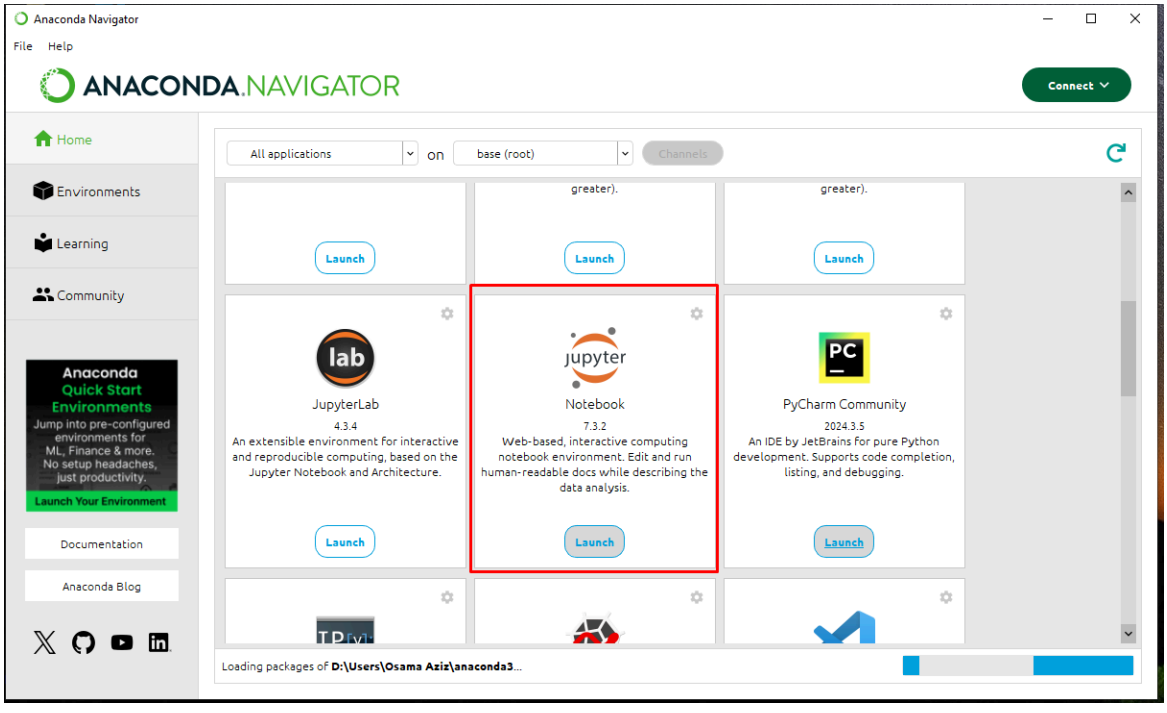
Skip registration

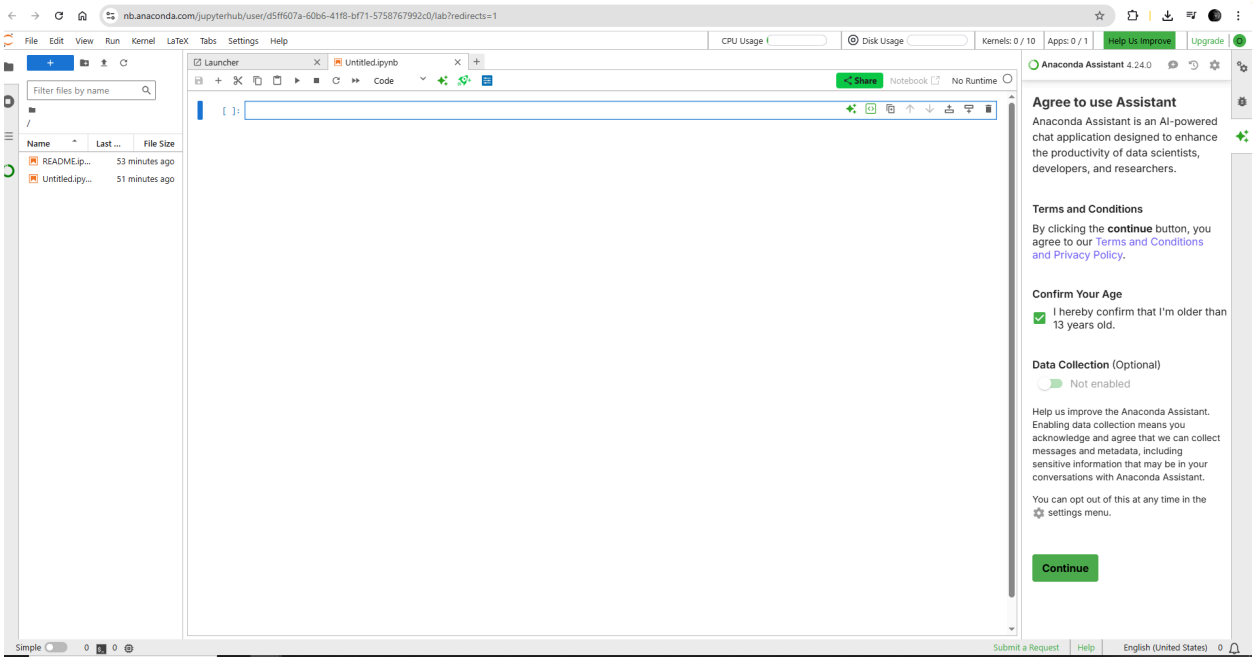
Manage Trusted Packages and Environments with Ease


anaconda.com/download/success
ANACONDA.
Products
Solutions
Resources
Company
Sign In
Get a Demo
Distribution Installers
Download
For installation assistance, refer to troubleshooting.
Windows
Mac
Linux
Miniconda Installers
Download
For installation assistance, refer to troubleshooting.
Windows
Mac
Linux
Hi, how can I help?

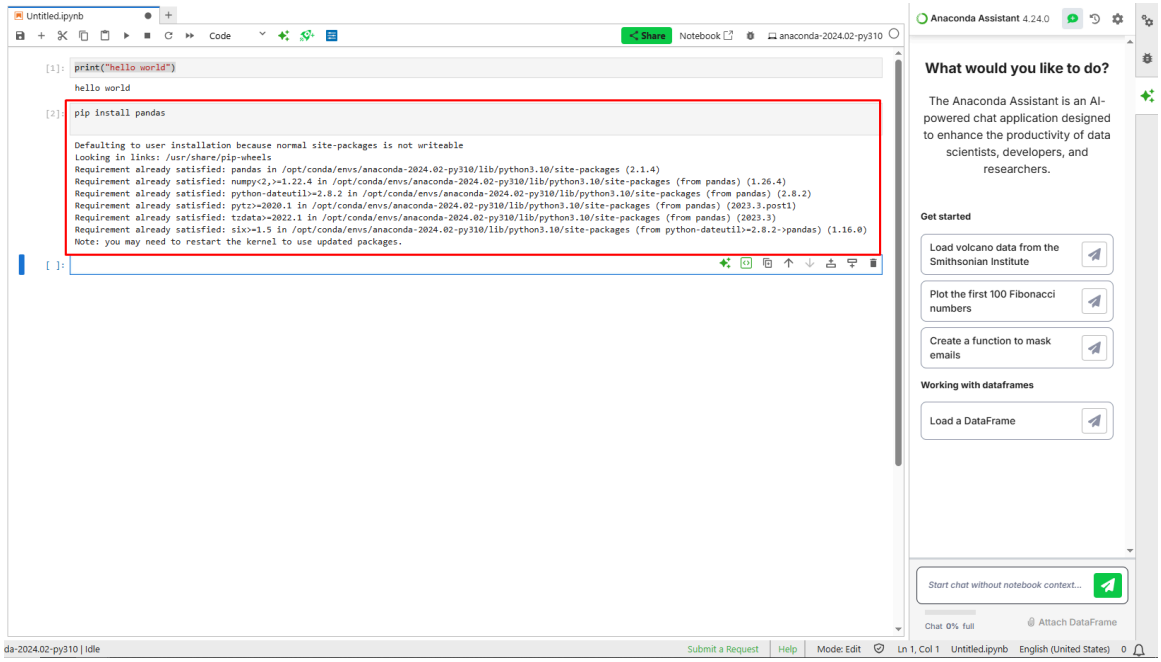

Anaconda3 2025.06-0 (64-bit) Setup
ANACONDA.
Installing
Please wait while Anaconda3 2025.06-0 (64-bit) is being installed.
Setting up the package cache...
Show details
Anaconda, Inc.
< Back
Next >
Cancel


Anaconda3 2025.06-0 (64-bit) Setup
ANACONDA.
Completing Anaconda3 2025.06-0 (64-bit) Setup
Thank you for installing Anaconda Distribution.
Here are some helpful resources to get you started. We recommend you bookmark the 'Getting Started with Anaconda Distribution' link so you can refer back to it later.
Launch Anaconda Navigator
Welcome to Anaconda
< Back
Finish
Cancel

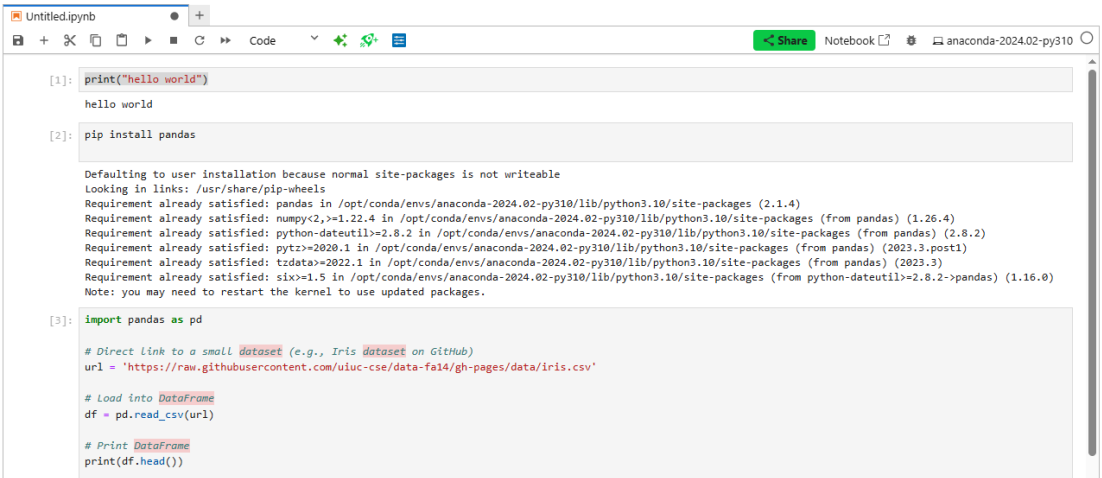

Anaconda Navigator
File Help
ANACONDA.NAVIGATOR
Connect
Home
Environments
Learning
Community
Anaconda Quick Start Environments
Jump into pre-configured environments for ML, Finance & more. No setup headaches, just productivity.
Launch Your Environment
Documentation
Anaconda Blog
All applications
on
base (root)
Channels
Launch
JupyterLab
4.3.4
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Notebook
7.3.2
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
PyCharm Community
2024.3.5
An IDE by JetBrains for pure Python development. Supports code completion, listing, and debugging.
Loading packages of D:\Users\Osama Aziz\anaconda3...



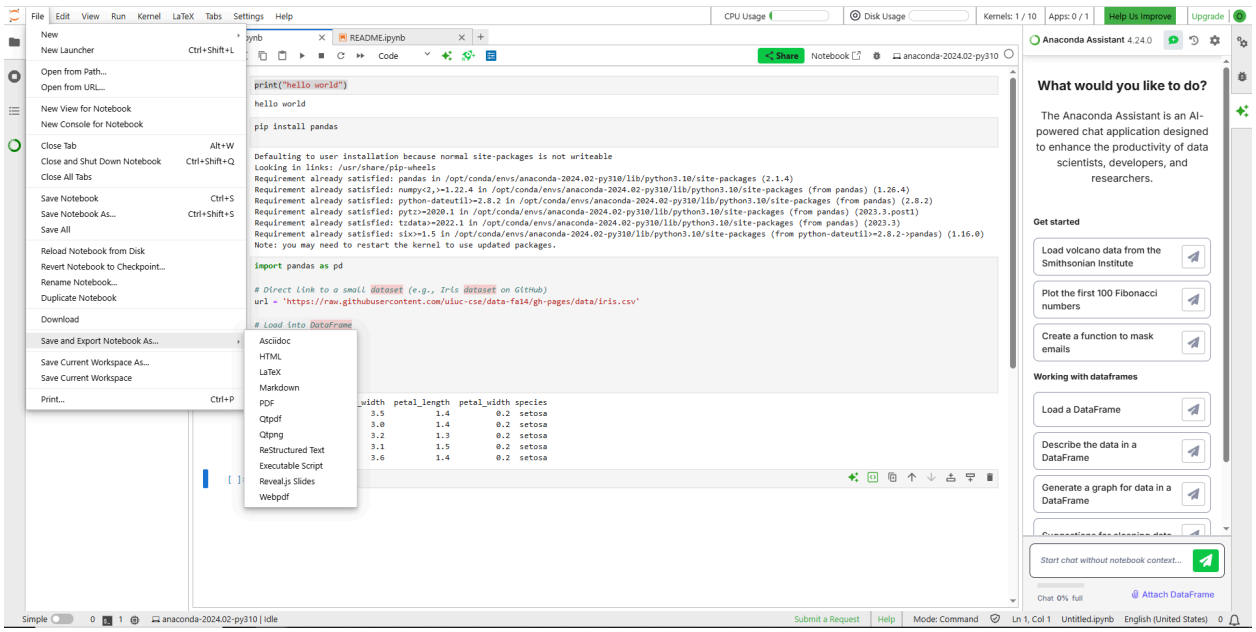
In Jupyter Notebook, you can write and execute Python code interactively, making it ideal for learning and data analysis. For example, you can use the `print()` function to display output, and install libraries like `pandas` using `pip install pandas`. With `pandas`, you can load datasets directly from a URL using `read_csv()` and view the data using functions like `head()`. This allows you to perform data handling and analysis efficiently within a single environment.



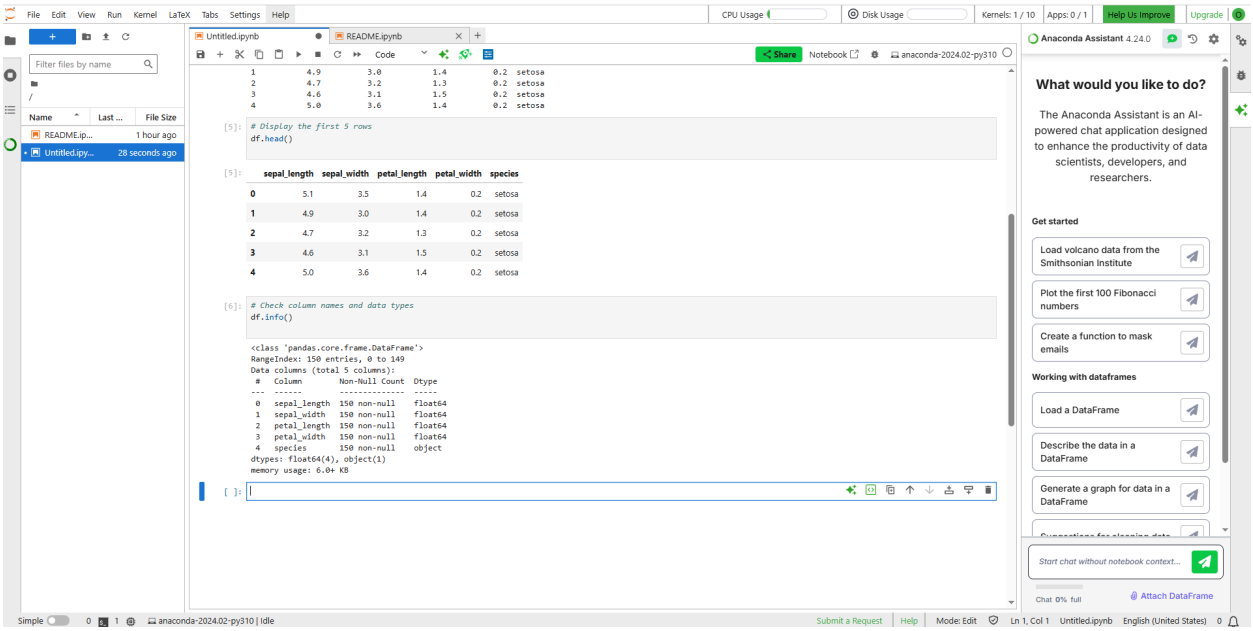
you can write and execute Python code interactively, which makes it well-suited for tasks such as testing commands, installing libraries, and working with data. For example, using the `pandas` library, you can read datasets directly from online sources with `read_csv()` and display them using `head()`. This environment allows you to combine code, output, and documentation in one place, streamlining data analysis and learning.



In Jupyter Notebook, once you have completed your work—such as importing libraries, loading datasets, and viewing results—you can save it by clicking **File > Save and Export Notebook As** and then choosing your preferred format (e.g., `.ipynb`, `.pdf`, `.html`). This ensures your code and output are preserved for future use or submission.



To quickly inspect a dataset in Jupyter Notebook, the `df.head()` function displays the first few rows (by default, five) of the DataFrame, which helps verify the structure and content of the data. Meanwhile, `df.info()` provides a concise summary including column names, data types, and the number of non-null entries, which is essential for understanding the dataset's composition and checking for missing values.



The `df.describe()` function in pandas generates descriptive statistics for all numerical columns in the DataFrame. It provides key summary metrics such as count, mean, standard deviation, minimum, maximum, and the 25th, 50th (median), and 75th percentiles. This helps in understanding the distribution, central tendency, and spread of the data at a glance.

```
[7]: df.describe()
```

[7]:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
[ ]:
```

6. Git and Version Controlling

A Story of Solving Problems Before They Start

Meet Amir, a computer science student who had just started working on his first major programming project—a web-based quiz system for his university. He was excited, writing code every day, testing new features, and making fast progress. But as the project grew, so did the complexity.

One day, Amir changed some parts of his code to add a new login feature. But after testing, he realized that not only was the new feature broken, but it also caused other parts of the program to stop working. He tried to undo his changes manually, but couldn't remember what the original code looked like. Frustrated, he wished he had saved a backup before editing.

Then, a friend introduced him to Git, a version control system. With Git, Amir could "commit" changes as he worked, like saving checkpoints in a game. Each commit represented a safe, restorable state of his code. Now, if something went wrong, he could easily go back to an earlier version. It was like having an unlimited undo button for his entire project.

Soon, Amir discovered GitHub, a platform that allowed him to store his Git projects online. Not only could he back up his work safely in the cloud, but he could also collaborate with classmates, track issues, and showcase his work publicly or privately.

Version control with Git—and using GitHub to manage and share code—completely changed the way Amir approached programming. He no longer feared breaking things while experimenting. Instead, he developed confidently, knowing that every change was tracked, reversible, and documented.

Just like Amir, every modern developer, from beginner to expert, uses version control to manage their code. It's not just about backing up—it's about working smarter, not harder.

Branches and Nodes in Git: Choosing Multiple Paths Without Losing the Way

As Amir became more comfortable with Git, he started experimenting with new features for his project—like adding a dark mode or a new grading algorithm. But each time he wanted to try something new, he worried it might break the existing, working version of the code.

That's when he learned about branches in Git.

He imagined his project like a tree. The main branch (called main or master) was the trunk—his stable, working version. But whenever he wanted to try something new, instead of changing the main code, he could create a branch, like a new limb on the tree. This new branch was a separate copy of his code where he could make changes freely, test ideas, or develop new features without affecting the main project.

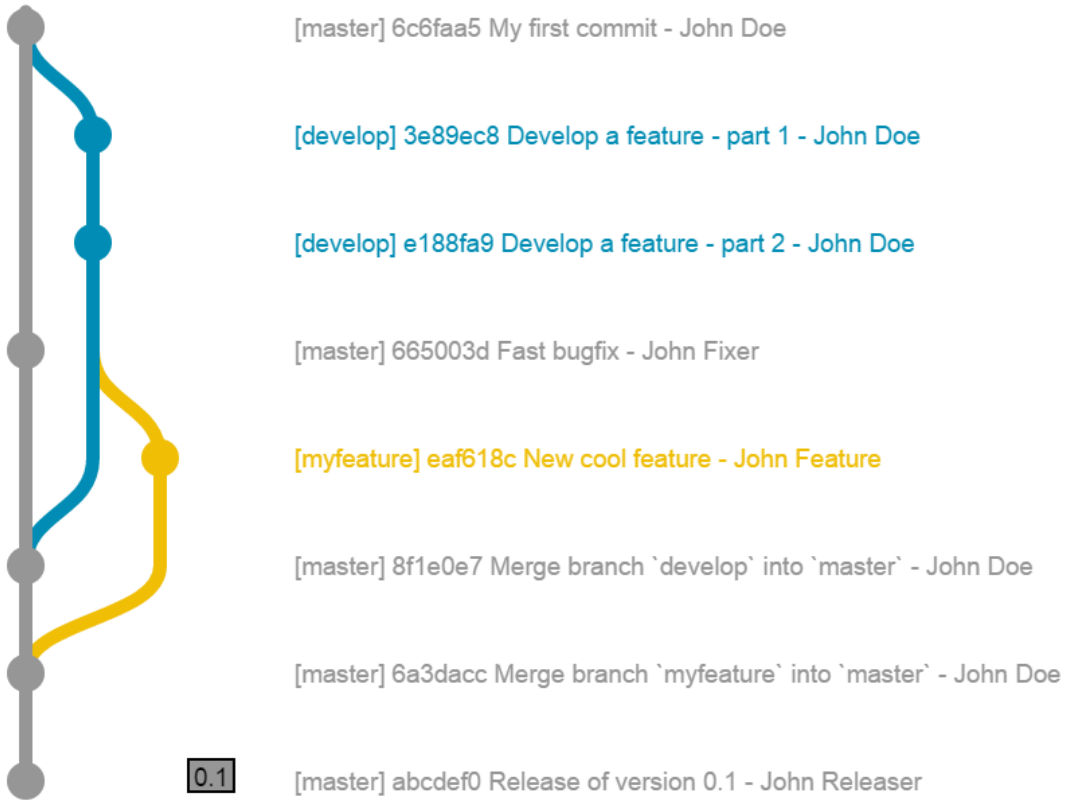
Each time he committed a change on any branch, it was stored as a node—a snapshot in the history. These nodes were connected like a timeline, showing exactly how the code had evolved. If his experiment worked, he could merge the branch back into the main trunk. If it didn't, he could simply delete the branch and the main project would remain untouched.

Amir found this especially useful when working with teammates. Each person could work on their own branch, and later all the branches could be reviewed and combined when ready. This way, collaboration became clean, organized, and safe.

Branches gave Amir the freedom to explore without fear, while nodes (commits) helped him keep track of every step.

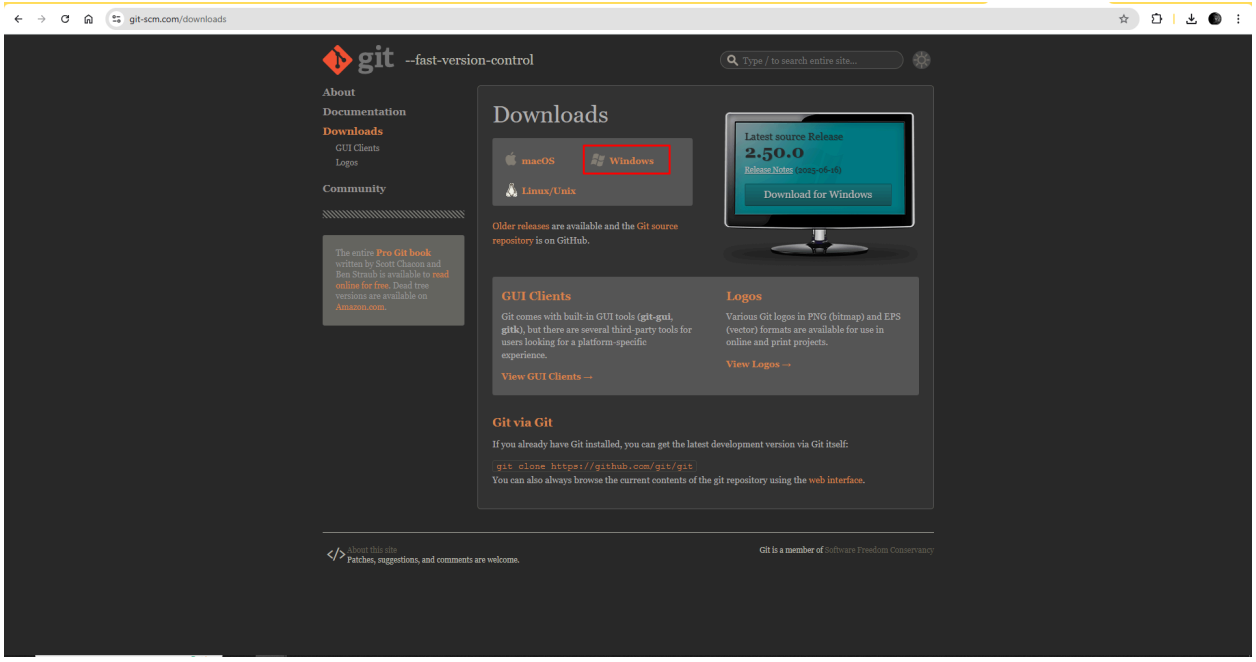
In Git and GitHub, branches help manage parallel development, and nodes (commits) keep a detailed record of progress—ensuring that nothing is ever truly lost.

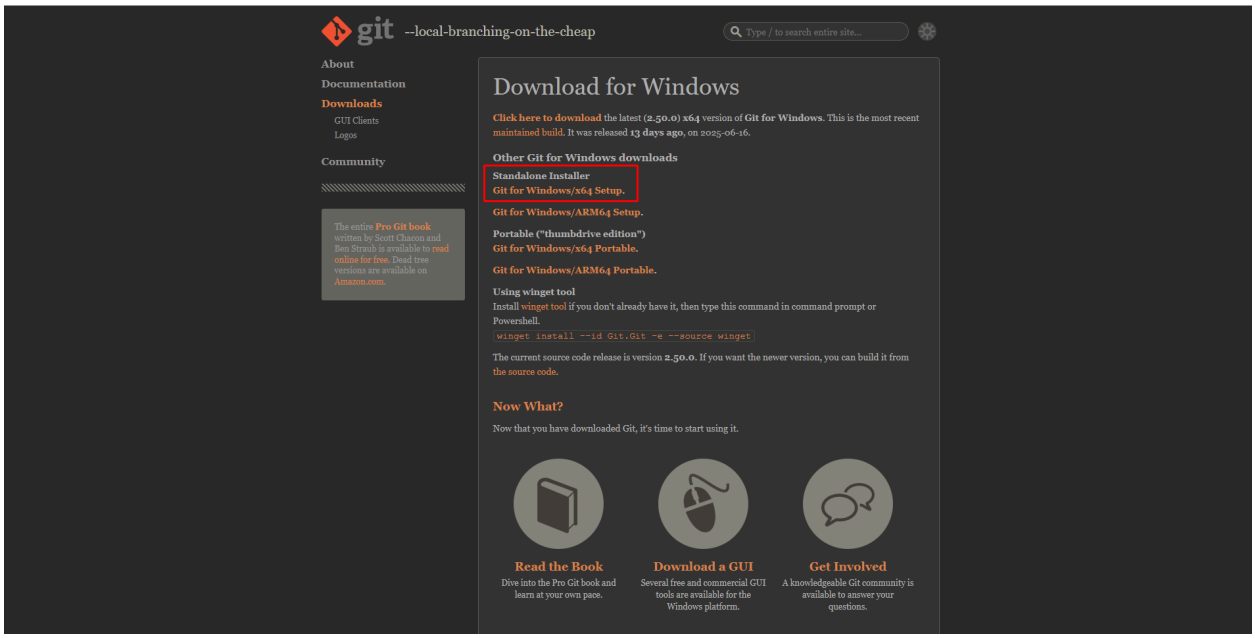
Let us see how this tree looks. In the real world we call this tree a git graph. Below is a Git history graph from a random GitHub project

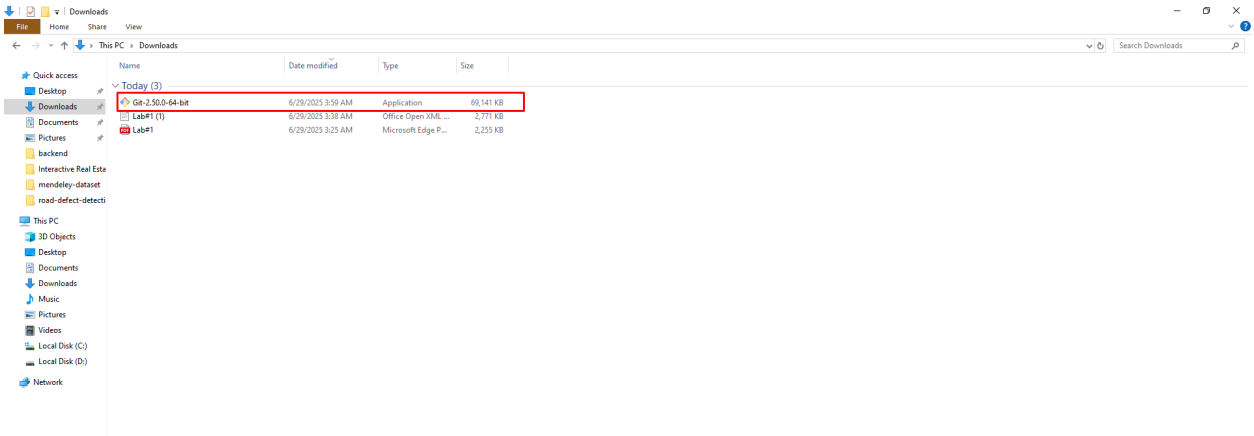


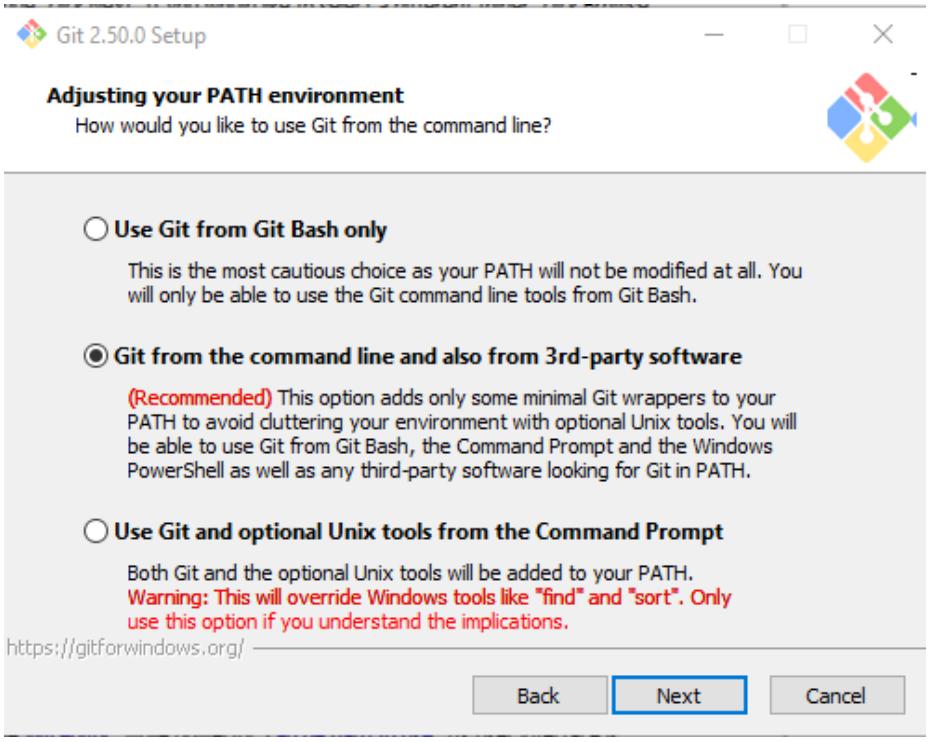
To install GIT on your systems, goto the link and follow the given steps.

https://git-scm.com/downloads

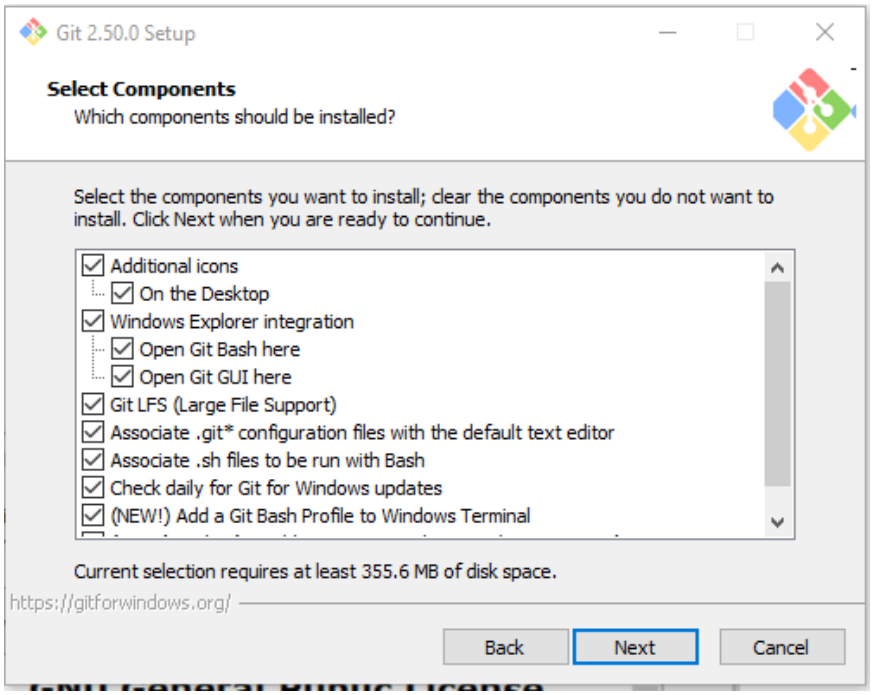


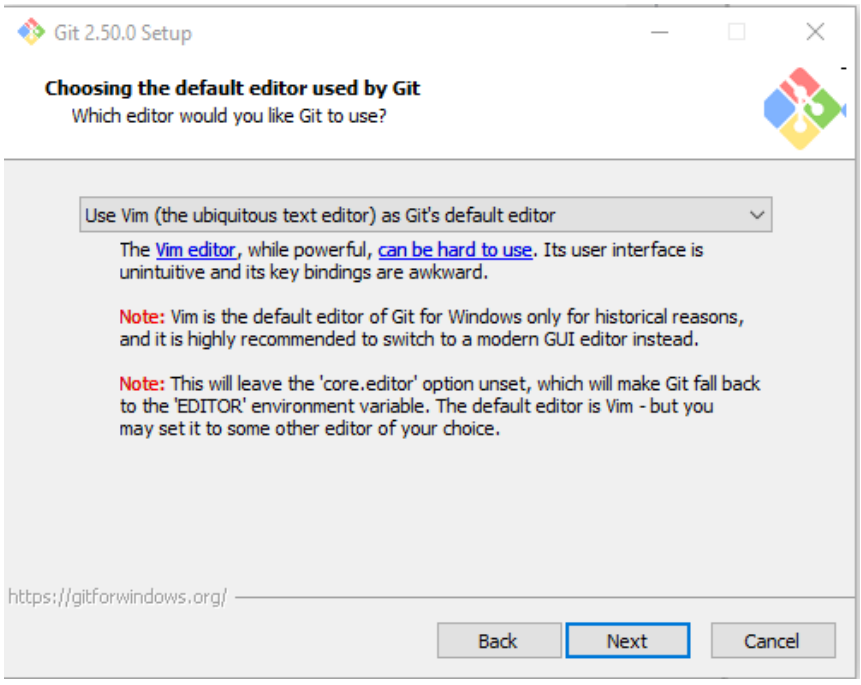

git --local-branching-on-the-cheap
Type / to search entire site...
About
Documentation
Downloads
GUI Clients
Logos
Community
The entire Pro Git book written by Scott Chacon and Ben Straub is available to read online for free. Dead tree versions are available on Amazon.com.
Download for Windows
Click here to download the latest (2.50.0) x64 version of Git for Windows. This is the most recent maintained build. It was released 13 days ago, on 2025-06-16.
Other Git for Windows downloads
Standalone Installer
Git for Windows/x64 Setup.
Git for Windows/ARM64 Setup.
Portable ("thumbdrive edition")
Git for Windows/x64 Portable.
Git for Windows/ARM64 Portable.
Using winget tool
Install winget tool if you don't already have it, then type this command in command prompt or Powershell.
winget install --id Git.Git -e --source winget
The current source code release is version 2.50.0. If you want the newer version, you can build it from the source code.
Now What?
Now that you have downloaded Git, it's time to start using it.
Read the Book
Dive into the Pro Git book and learn at your own pace.
Download a GUI
Several free and commercial GUI tools are available for the Windows platform.
Get Involved
A knowledgeable Git community is available to answer your questions.

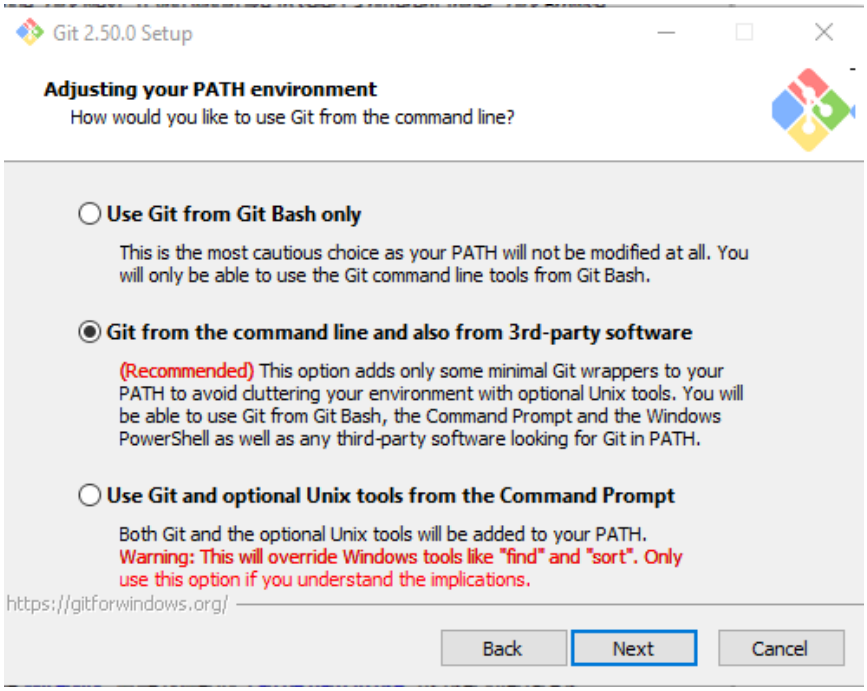

Downloads
File
Home
Share
View
This PC › Downloads
Search Downloads
Quick access
Desktop
Downloads
Documents
Pictures
backend
Interactive Real Esta
mendeley-dataset
road-defect-detecti
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Local Disk (C:)
Local Disk (D:)
Network
Name
Date modified
Type
Size
Today (3)
Git-2.50.0-64-bit 6/29/2025 3:59 AM Application 69,141 KB
Lab#1 (1) 6/29/2025 3:38 AM Office Open XML ... 2,771 KB
Lab#1 6/29/2025 3:25 AM Microsoft Edge P... 2,255 KB

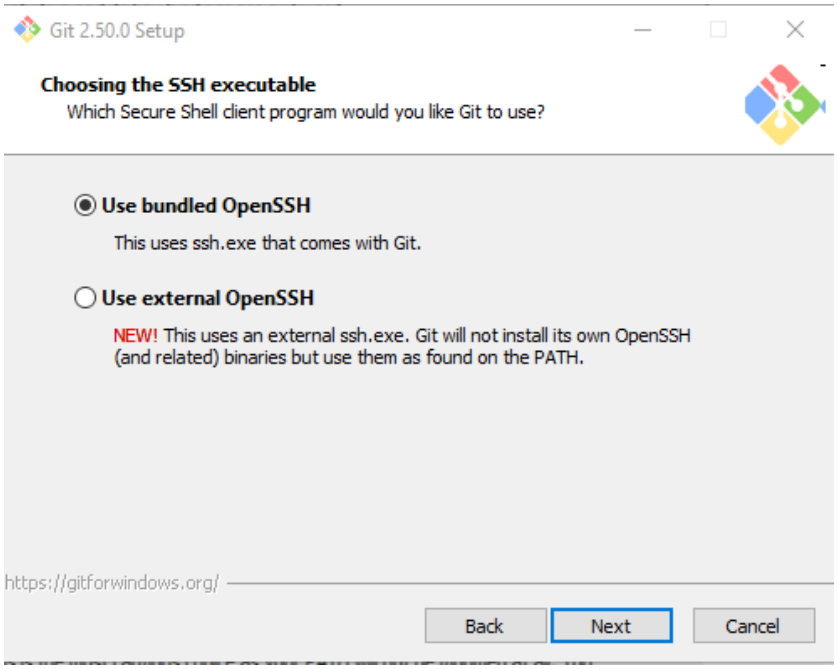

Git 2.50.0 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

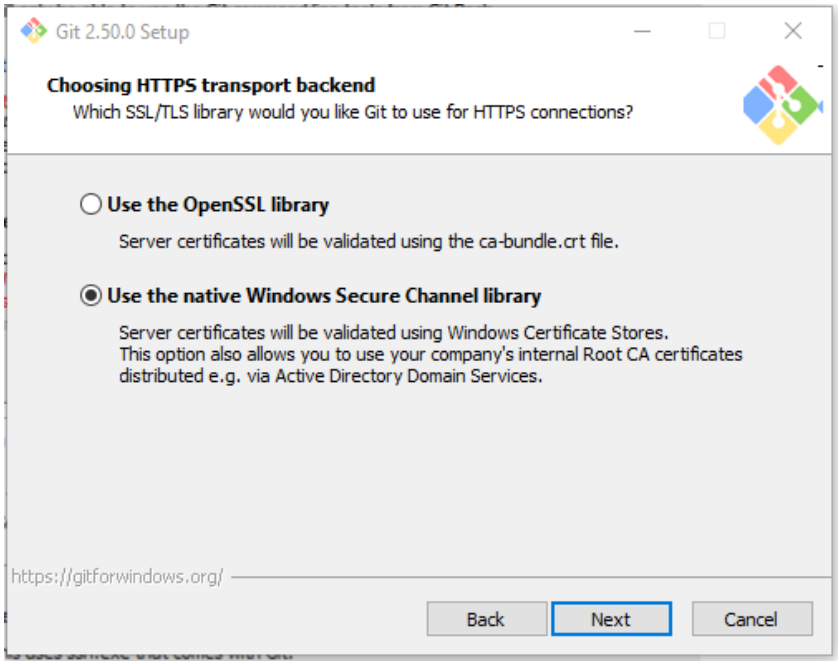

Git 2.50.0 Setup
Information
Please read the following important information before continuing.
When you are ready to continue with Setup, click Next.
GNU General Public License
Version 2, June 1991
Copyright (C) 1989, 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330, Boston, MA 02111-1307, USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.
Preamble
The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change
https://gitforwindows.org/
Next
Cancel

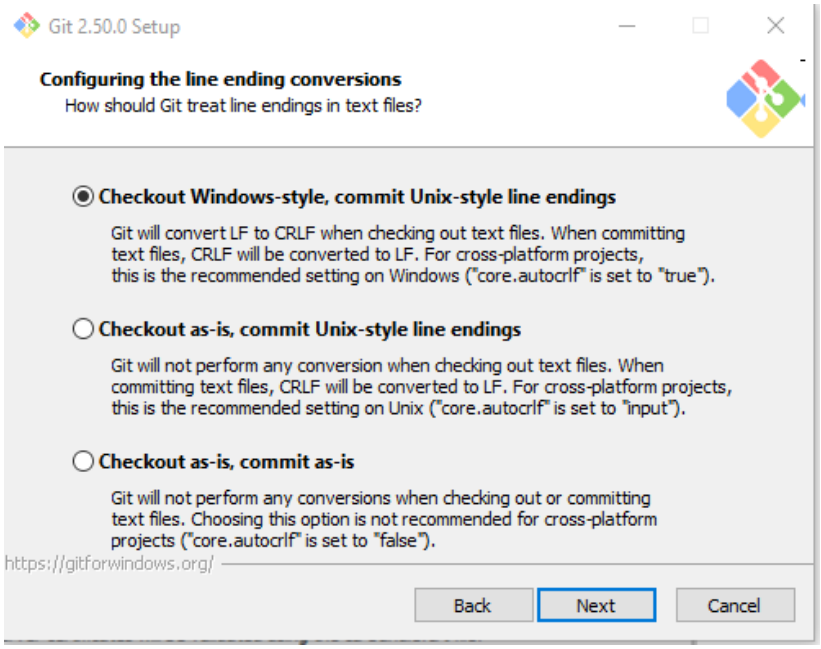

Git 2.50.0 Setup
Select Components
Which components should be installed?
Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.
Additional icons
On the Desktop
Windows Explorer integration
Open Git Bash here
Open Git GUI here
Git LFS (Large File Support)
Associate .git* configuration files with the default text editor
Associate .sh files to be run with Bash
Check daily for Git for Windows updates
(NEW!) Add a Git Bash Profile to Windows Terminal
Current selection requires at least 355.6 MB of disk space.
https://gitforwindows.org/
Back
Next
Cancel

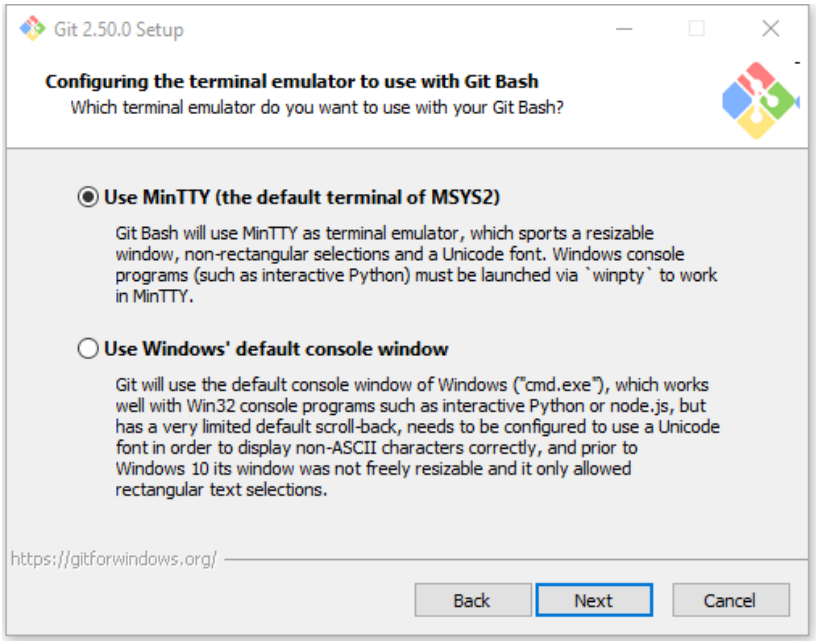

Git 2.50.0 Setup
Choosing the default editor used by Git
Which editor would you like Git to use?
Use Vim (the ubiquitous text editor) as Git's default editor
The Vim editor, while powerful, can be hard to use. Its user interface is unintuitive and its key bindings are awkward.
Note: Vim is the default editor of Git for Windows only for historical reasons, and it is highly recommended to switch to a modern GUI editor instead.
Note: This will leave the 'core.editor' option unset, which will make Git fall back to the 'EDITOR' environment variable. The default editor is Vim - but you may set it to some other editor of your choice.
https://gitforwindows.org/
Back
Next
Cancel

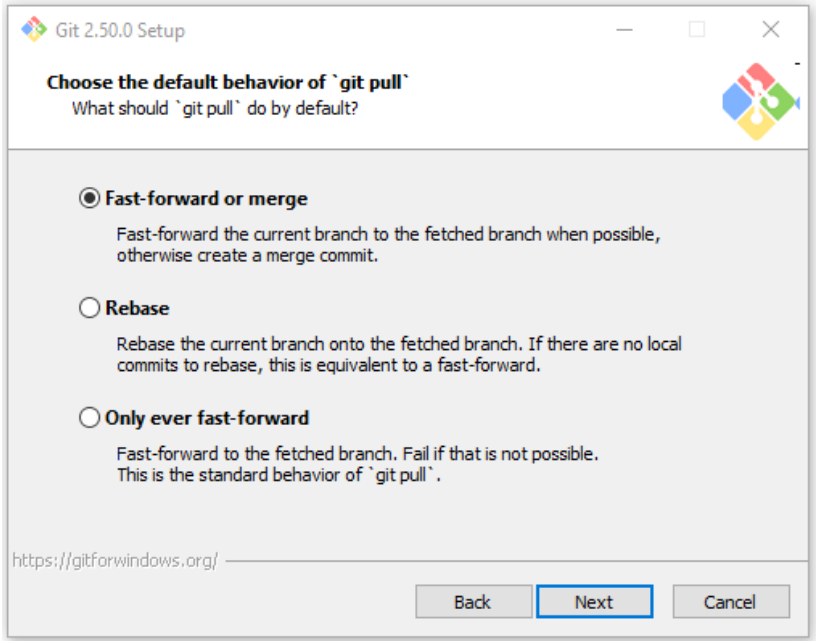

Git 2.50.0 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

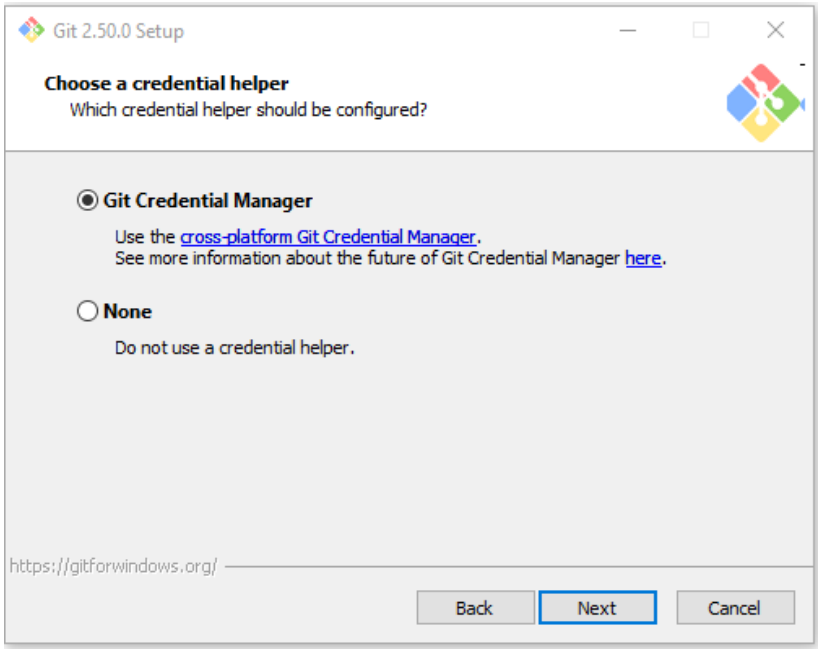

Git 2.50.0 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use external OpenSSH
NEW! This uses an external ssh.exe. Git will not install its own OpenSSH (and related) binaries but use them as found on the PATH.
https://gitforwindows.org/
Back
Next
Cancel

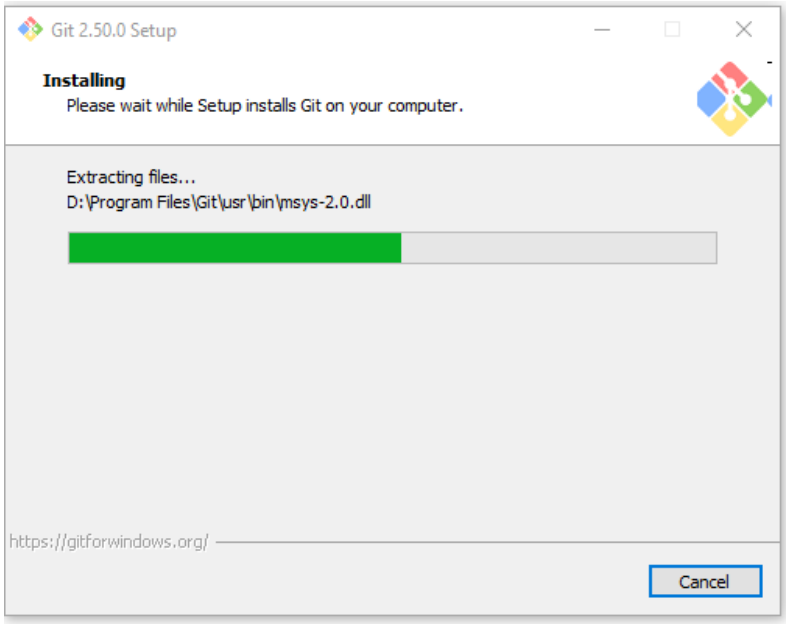

Git 2.50.0 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates
distributed e.g. via Active Directory Domain Services.
https://gitforwindows.org/
Back
Next
Cancel

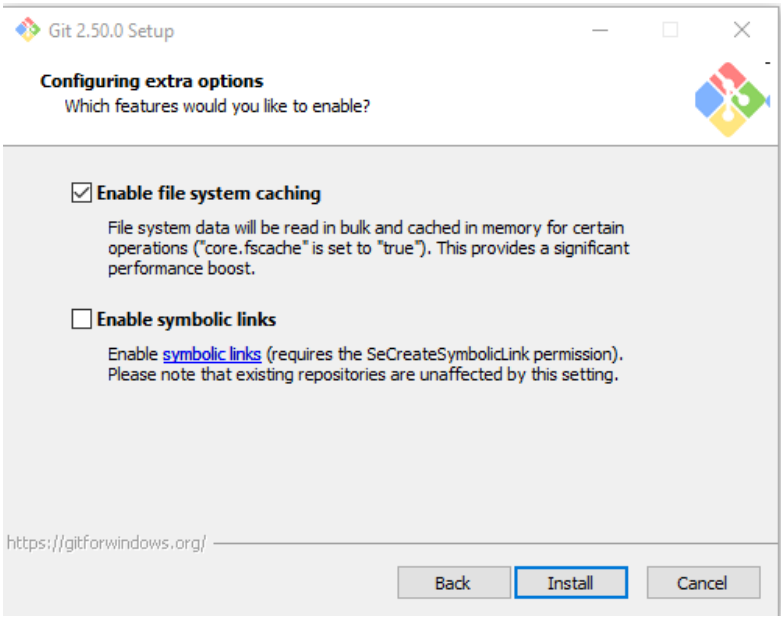

Git 2.50.0 Setup
Configuring the line ending conversions
How should Git treat line endings in text files?
Checkout Windows-style, commit Unix-style line endings
Git will convert LF to CRLF when checking out text files. When committing
text files, CRLF will be converted to LF. For cross-platform projects,
this is the recommended setting on Windows ("core.autocrlf" is set to "true").
Checkout as-is, commit Unix-style line endings
Git will not perform any conversion when checking out text files. When
committing text files, CRLF will be converted to LF. For cross-platform projects,
this is the recommended setting on Unix ("core.autocrlf" is set to "input").
Checkout as-is, commit as-is
Git will not perform any conversions when checking out or committing
text files. Choosing this option is not recommended for cross-platform
projects ("core.autocrlf" is set to "false").
https://gitforwindows.org/
Back
Next
Cancel

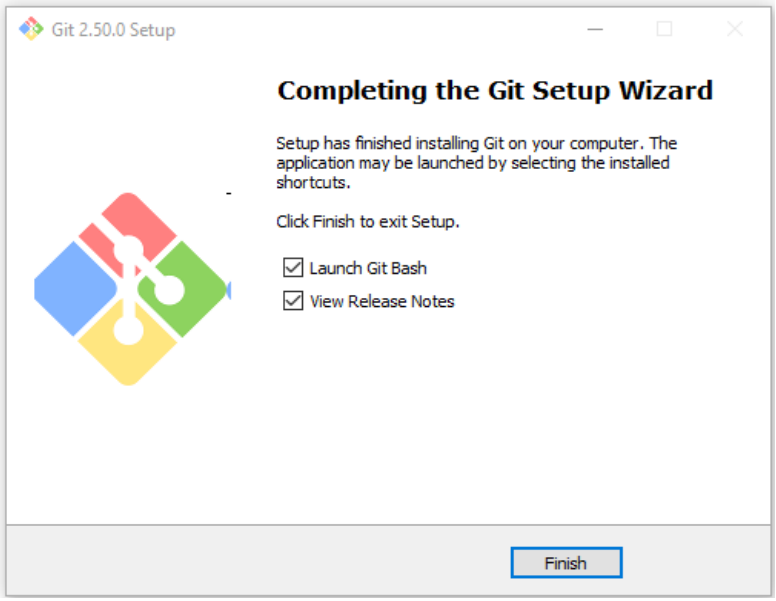

Git 2.50.0 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://gitforwindows.org/
Back
Next
Cancel

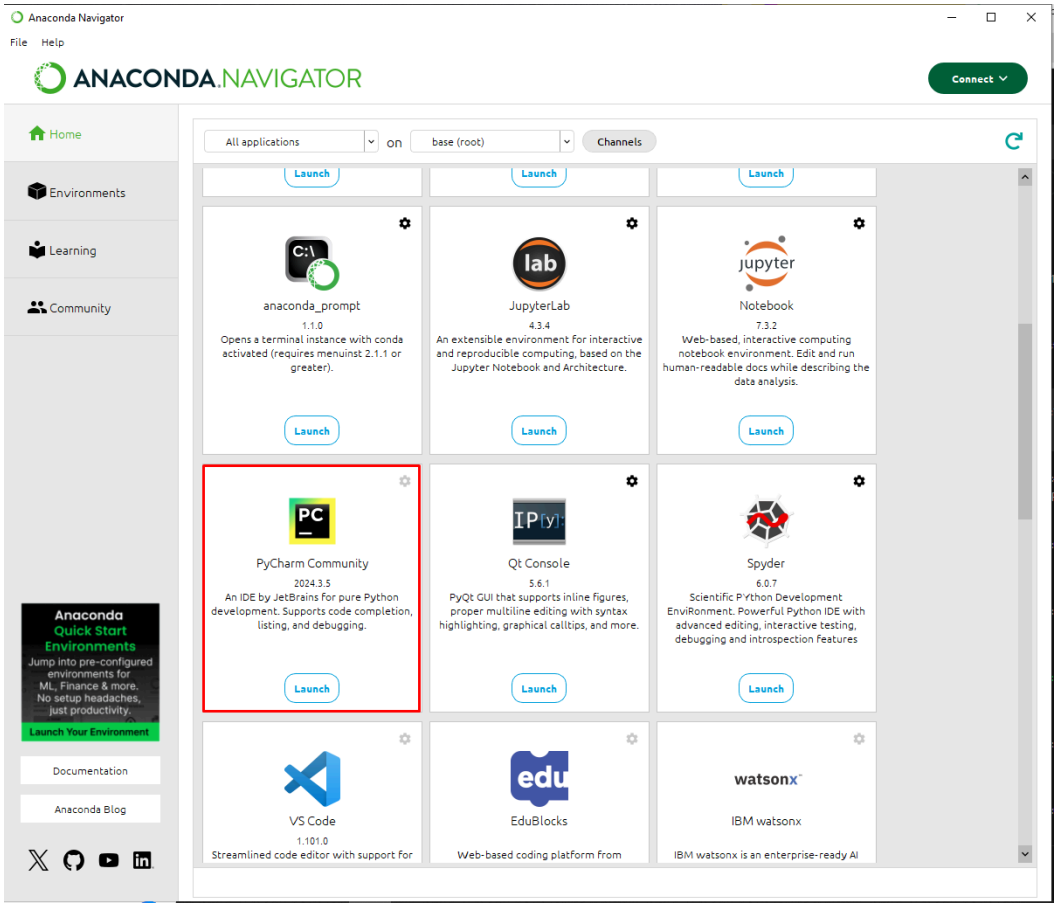

Git 2.50.0 Setup
Choose the default behavior of `git pull`
What should `git pull` do by default?
Fast-forward or merge
Fast-forward the current branch to the fetched branch when possible, otherwise create a merge commit.
Rebase
Rebase the current branch onto the fetched branch. If there are no local commits to rebase, this is equivalent to a fast-forward.
Only ever fast-forward
Fast-forward to the fetched branch. Fail if that is not possible. This is the standard behavior of `git pull`.
https://gitforwindows.org/
Back
Next
Cancel

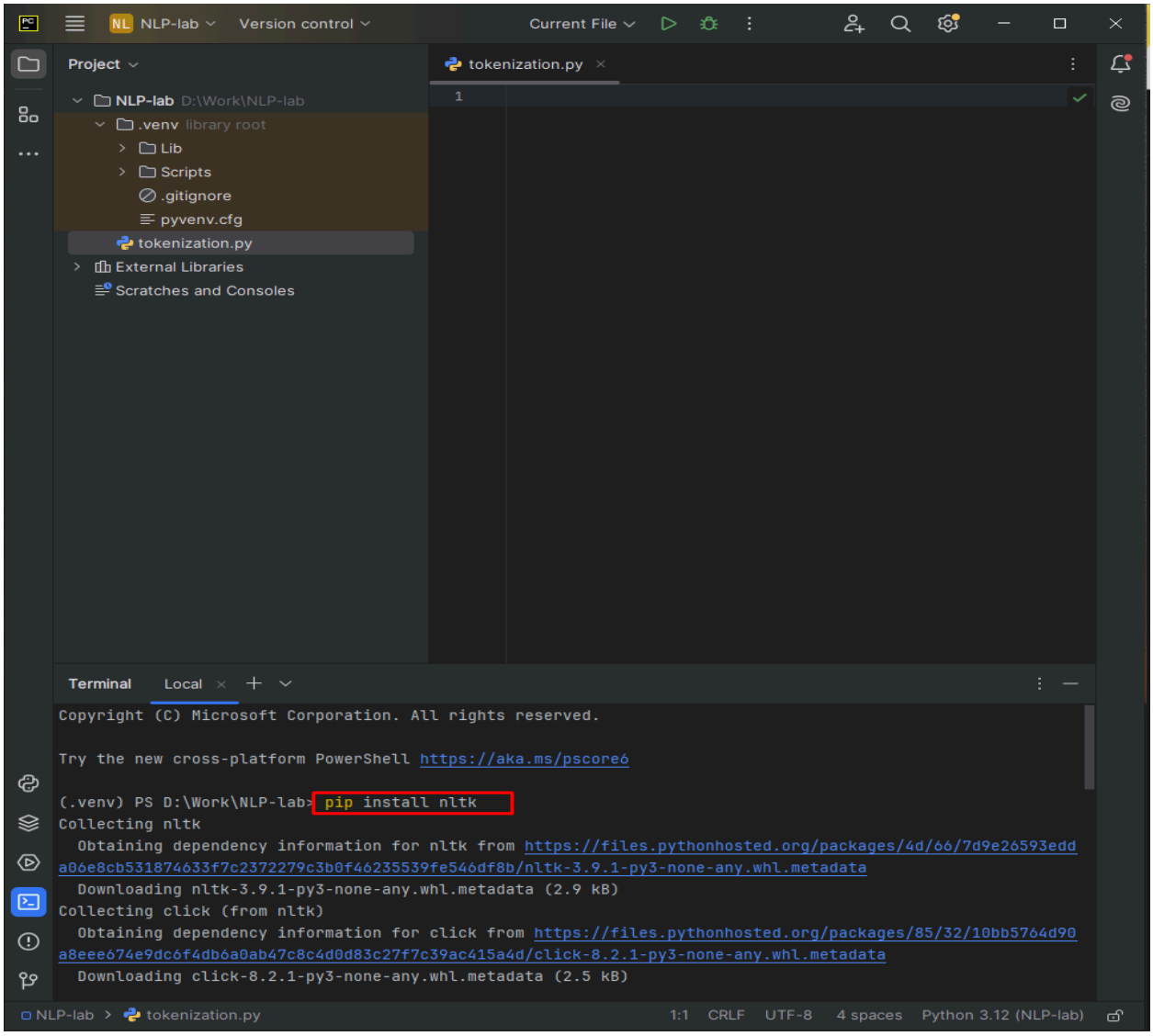

Git 2.50.0 Setup
Choose a credential helper
Which credential helper should be configured?
Git Credential Manager
Use the cross-platform Git Credential Manager.
See more information about the future of Git Credential Manager here.
None
Do not use a credential helper.
https://gitforwindows.org/
Back
Next
Cancel

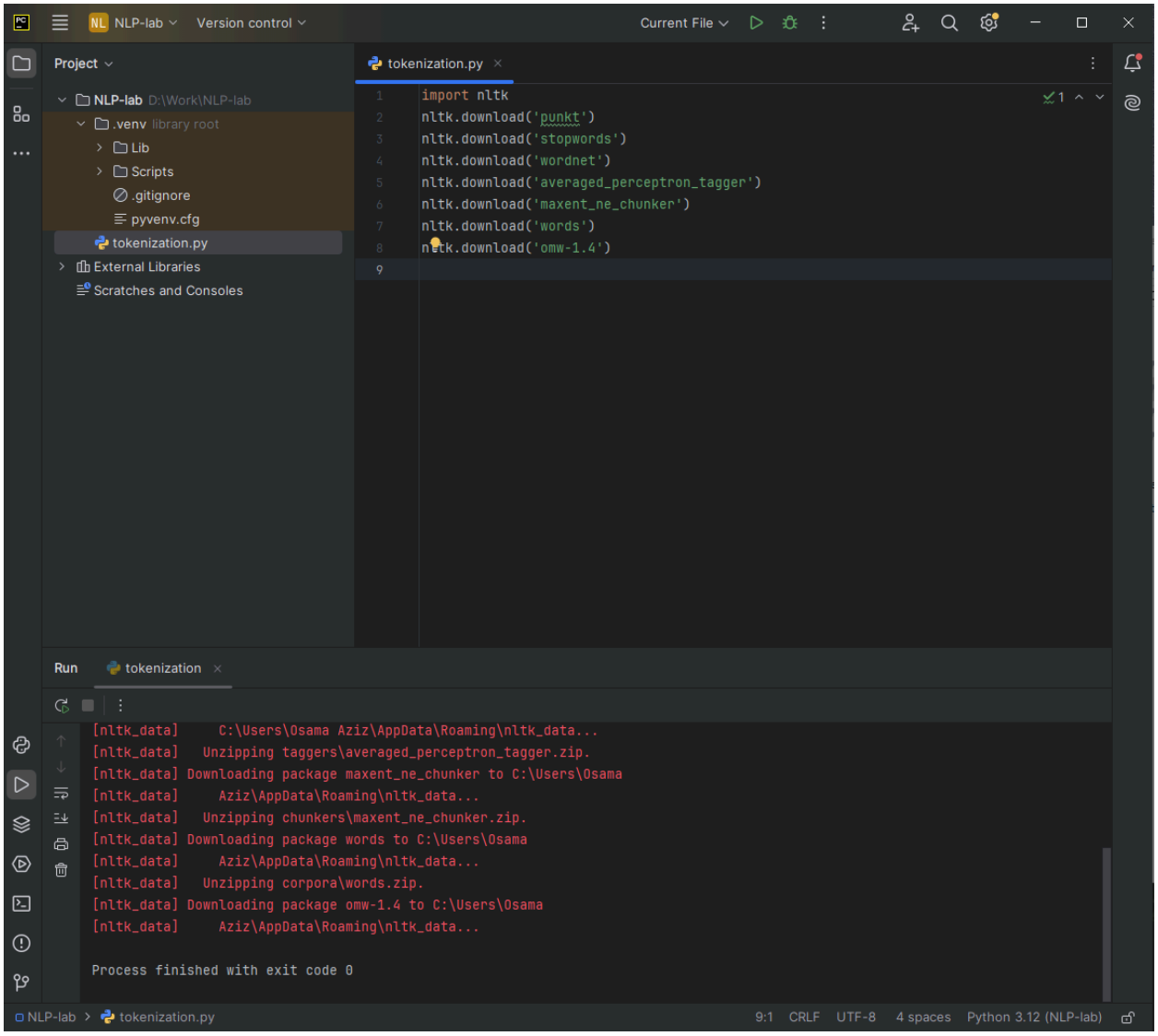

Git 2.50.0 Setup
Installing
Please wait while Setup installs Git on your computer.
Extracting files...
D:\Program Files\Git\usr\bin\msys-2.0.dll
https://gitforwindows.org/
Cancel

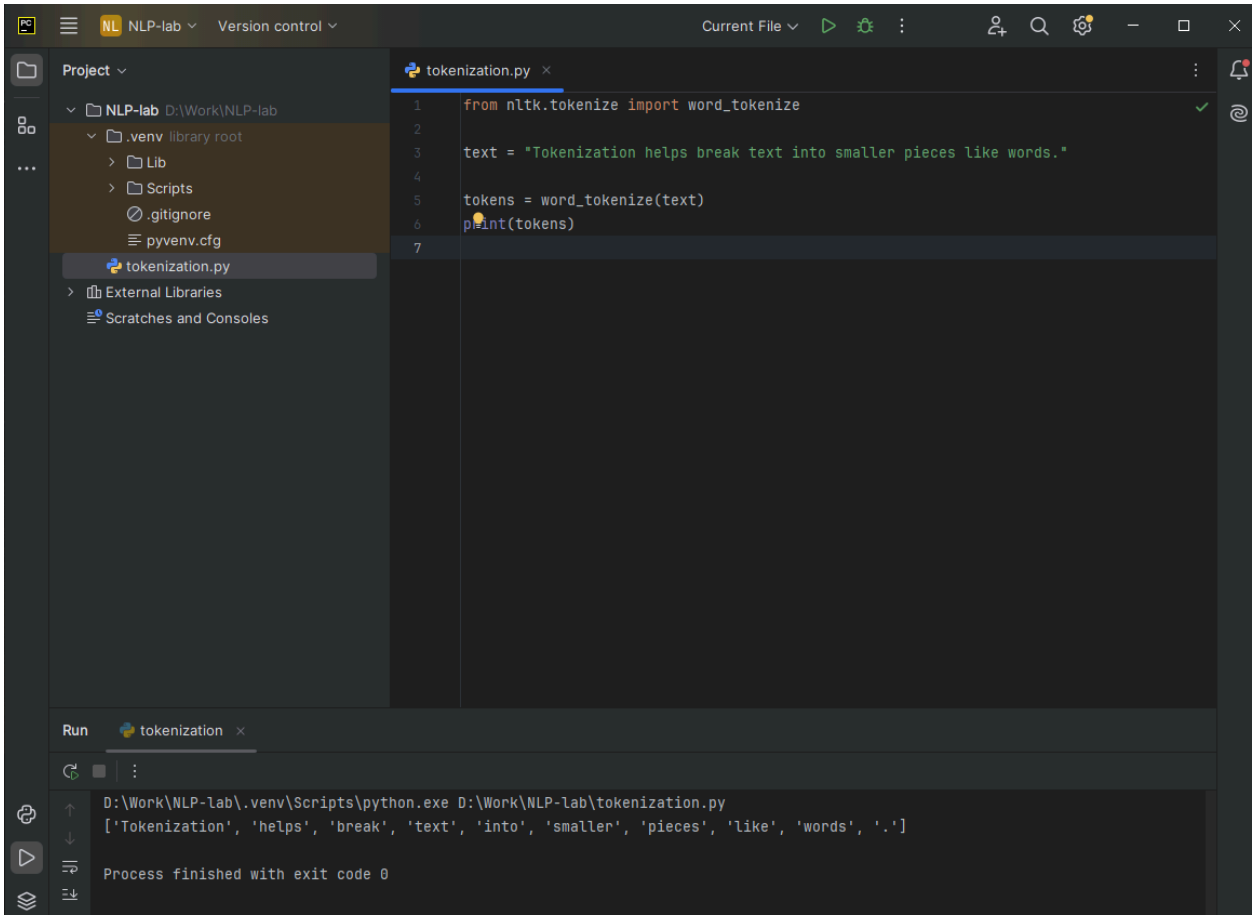

Git 2.50.0 Setup
Configuring extra options
Which features would you like to enable?
Enable file system caching
File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant performance boost.
Enable symbolic links
Enable symbolic links (requires the SeCreateSymbolicLink permission). Please note that existing repositories are unaffected by this setting.
https://gitforwindows.org/
Back
Install
Cancel

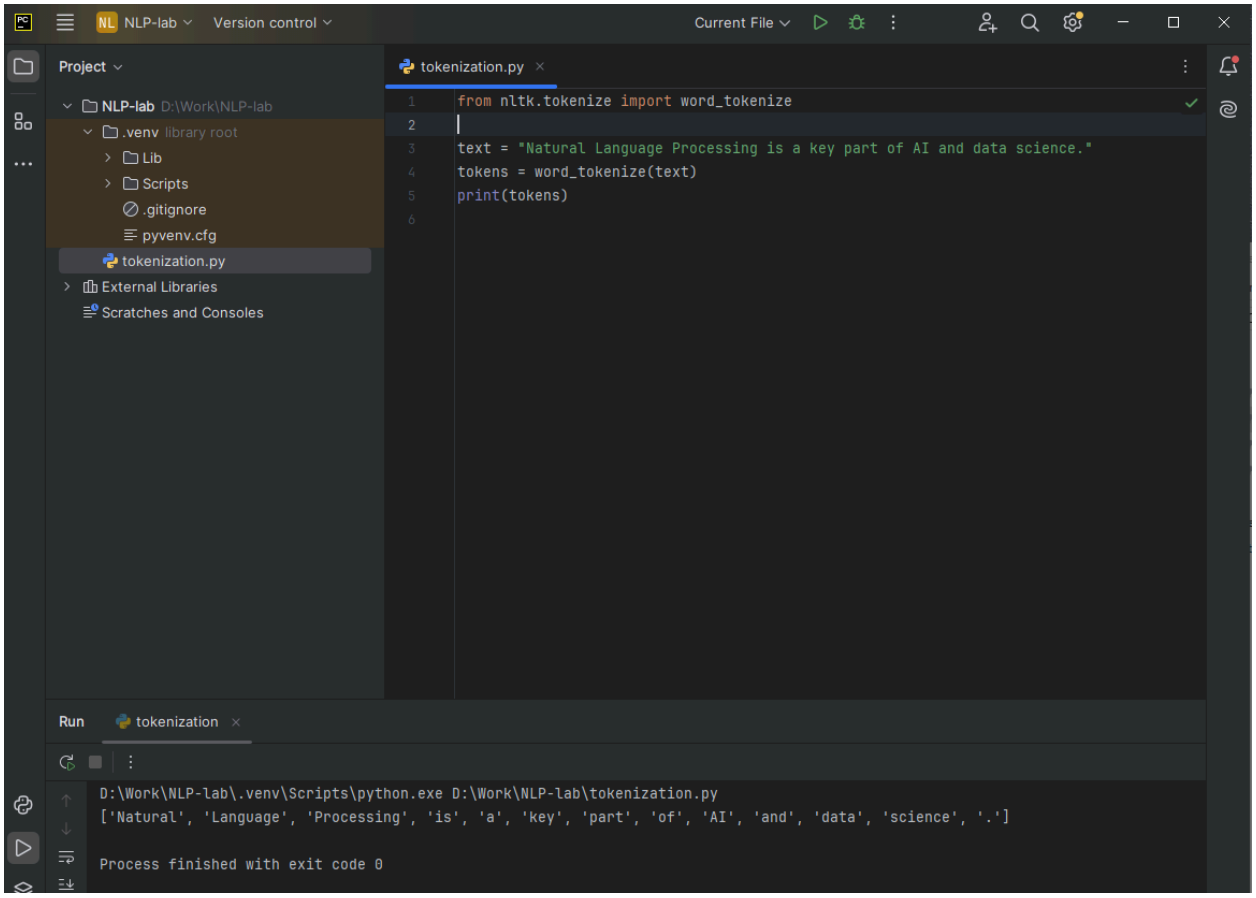

Git 2.50.0 Setup
Completing the Git Setup Wizard
Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch Git Bash
View Release Notes
Finish

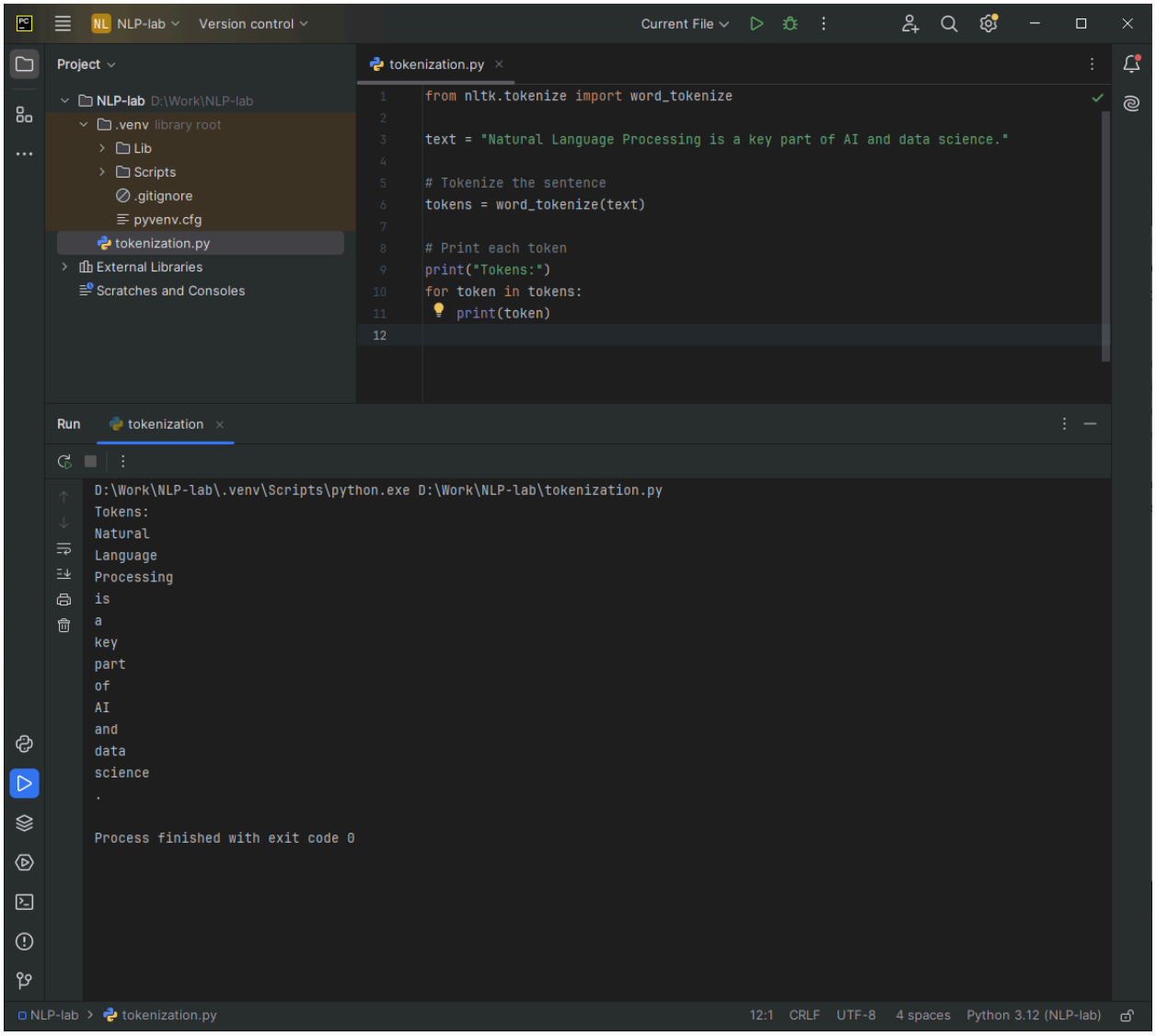

Anaconda Navigator
File Help
ANACONDA.NAVIGATOR
Connect
Home
Environments
Learning
Community
All applications
on
base (root)
Channels
Launch
anaconda_prompt
1.1.0
Opens a terminal instance with conda activated (requires menuinst 2.1.1 or greater).
JupyterLab
4.3.4
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Notebook
7.3.2
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
PyCharm Community
2024.3.5
An IDE by JetBrains for pure Python development. Supports code completion, listing, and debugging.
Qt Console
5.6.1
PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.
Spyder
6.0.7
Scientific PYthon Development EnviRonment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features
VS Code
1.101.0
Streamlined code editor with support for
EduBlocks
Web-based coding platform from
IBM watsonx
IBM watsonx is an enterprise-ready AI
Anaconda
Quick Start
Environments
Jump into pre-configured environments for ML, Finance & more. No setup headaches, just productivity.
Launch Your Environment
Documentation
Anaconda Blog

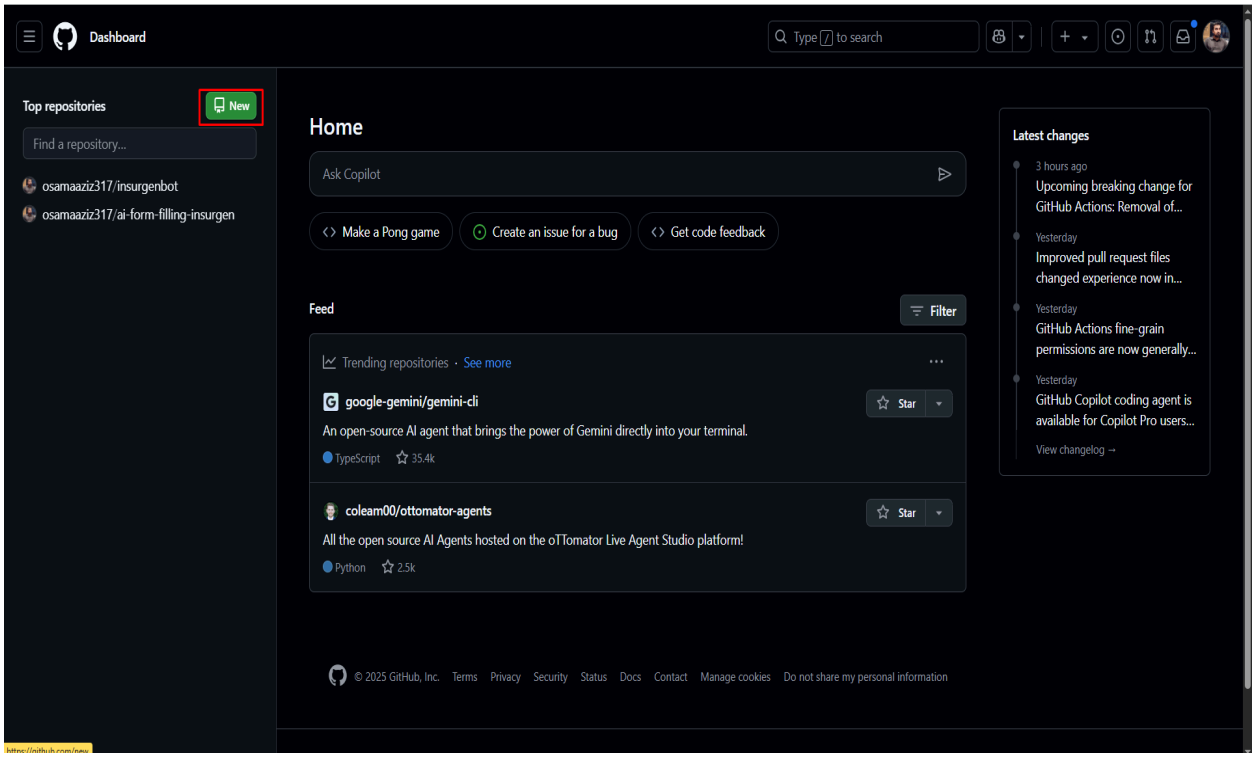

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
1
Terminal
Local
Copyright (C) Microsoft Corporation. All rights reserved.
Try the new cross-platform PowerShell https://aka.ms/pscore6
(.venv) PS D:\Work\NLP-lab> pip install nltk
Collecting nltk
Obtaining dependency information for nltk from https://files.pythonhosted.org/packages/4d/66/7d9e26593edda06e8cb531874633f7c2372279c3b0f46235539fe546df8b/nltk-3.9.1-py3-none-any.whl.metadata
Downloading nltk-3.9.1-py3-none-any.whl.metadata (2.9 kB)
Collecting click (from nltk)
Obtaining dependency information for click from https://files.pythonhosted.org/packages/85/32/10bb5764d90a8eee674e9dc6f4db6a0ab47c8c4d0d83c27f7c39ac415a4d/click-8.2.1-py3-none-any.whl.metadata
Downloading click-8.2.1-py3-none-any.whl.metadata (2.5 kB)
NLP-lab > tokenization.py
1:1
CRLF
UTF-8
4 spaces
Python 3.12 (NLP-lab)

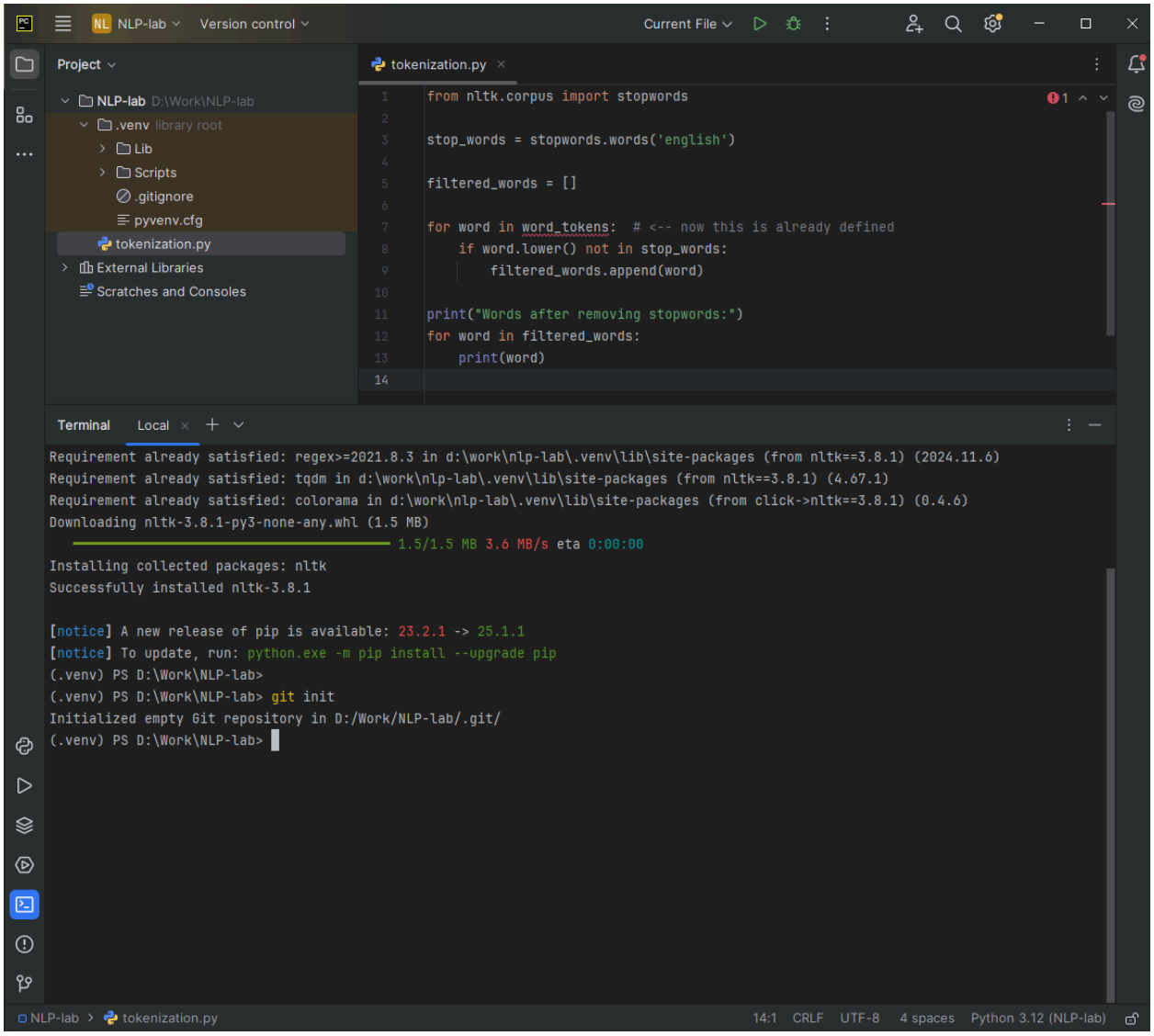

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('omw-1.4')
Run
tokenization
[nltk_data] C:\Users\Osama Aziz\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package maxent_ne_chunker to C:\Users\Osama
[nltk_data] Aziz\AppData\Roaming\nltk_data...
[nltk_data] Unzipping chunkers\maxent_ne_chunker.zip.
[nltk_data] Downloading package words to C:\Users\Osama
[nltk_data] Aziz\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\words.zip.
[nltk_data] Downloading package omw-1.4 to C:\Users\Osama
[nltk_data] Aziz\AppData\Roaming\nltk_data...
Process finished with exit code 0
NLP-lab
tokenization.py
9:1
CRLF
UTF-8
4 spaces
Python 3.12 (NLP-lab)

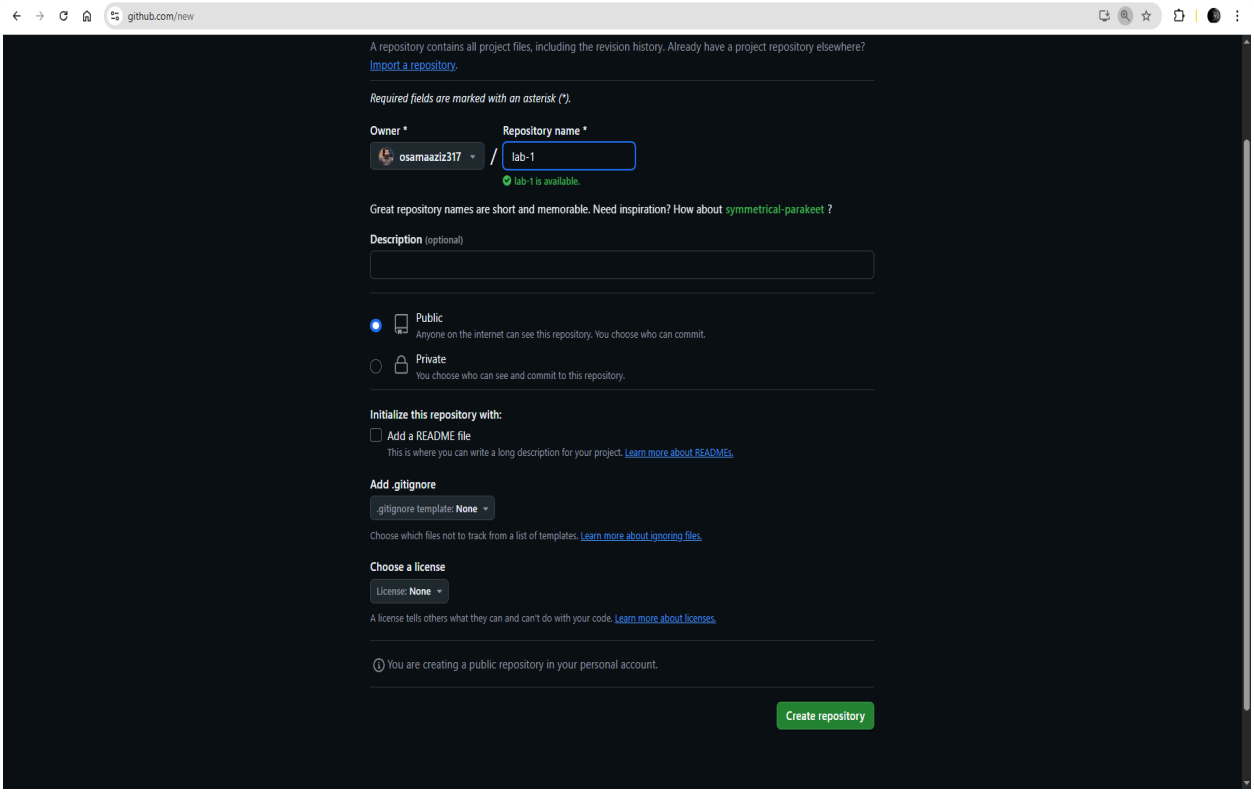

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
1 from nltk.tokenize import word_tokenize
2
3 text = "Tokenization helps break text into smaller pieces like words."
4
5 tokens = word_tokenize(text)
6 print(tokens)
7
Run
tokenization
D:\Work\NLP-lab\.venv\Scripts\python.exe D:\Work\NLP-lab\tokenization.py
['Tokenization', 'helps', 'break', 'text', 'into', 'smaller', 'pieces', 'like', 'words', '.']
Process finished with exit code 0

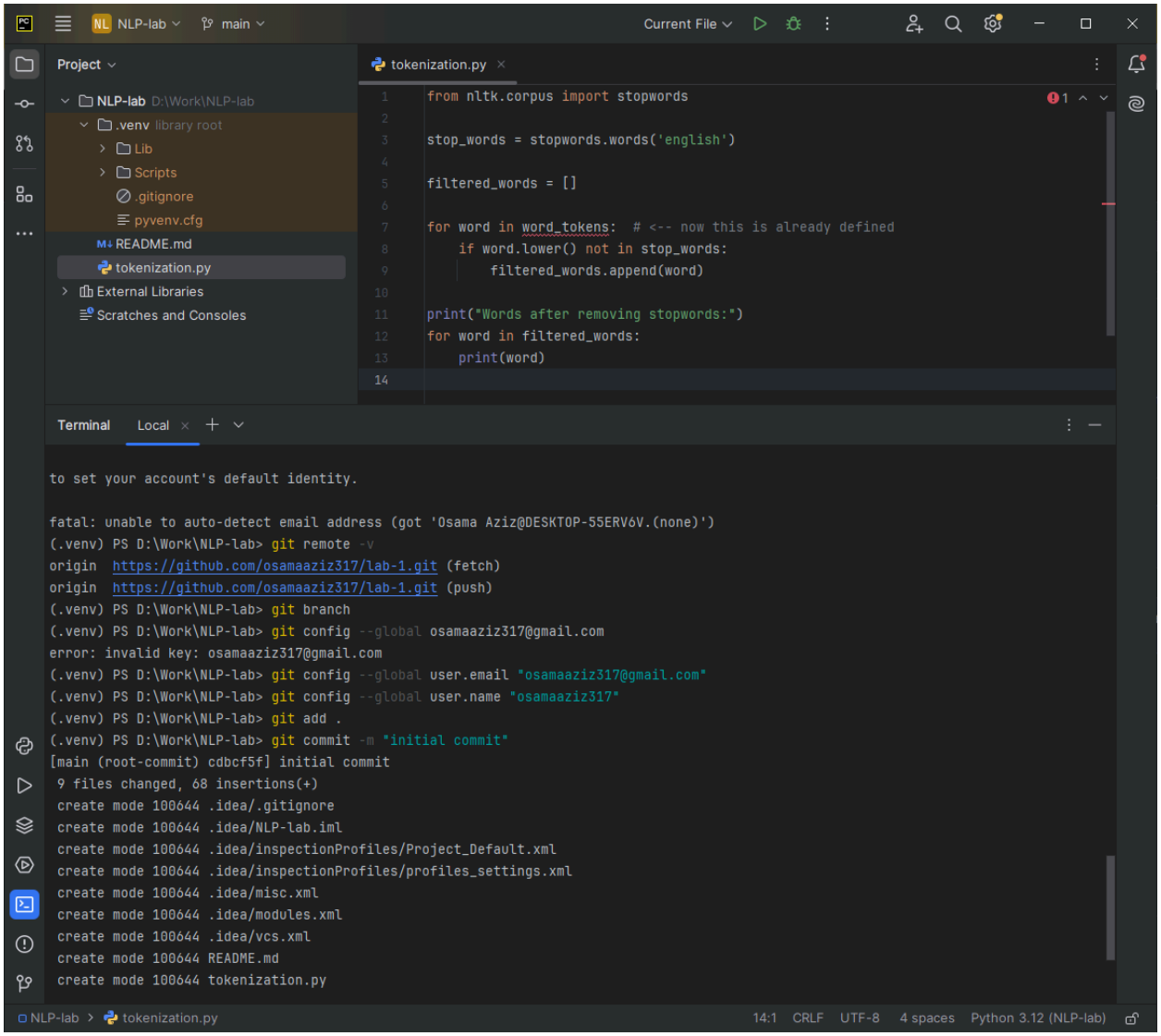

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
1 from nltk.tokenize import word_tokenize
2
3 text = "Natural Language Processing is a key part of AI and data science."
4 tokens = word_tokenize(text)
5 print(tokens)
6
Run
tokenization
D:\Work\NLP-lab\.venv\Scripts\python.exe D:\Work\NLP-lab\tokenization.py
['Natural', 'Language', 'Processing', 'is', 'a', 'key', 'part', 'of', 'AI', 'and', 'data', 'science', '.']
Process finished with exit code 0

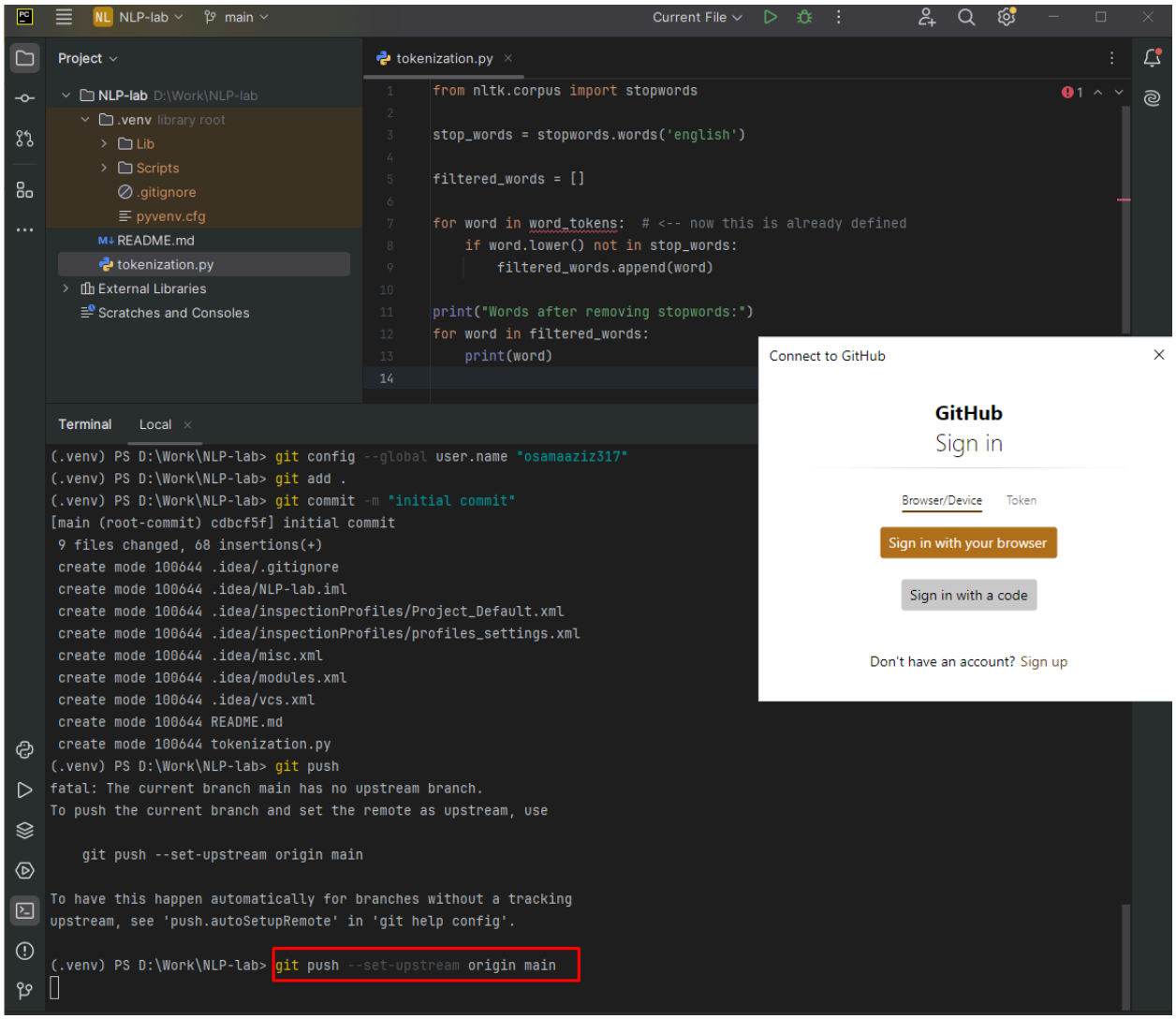

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
from nltk.tokenize import word_tokenize
text = "Natural Language Processing is a key part of AI and data science."
Tokenize the sentence
tokens = word_tokenize(text)
Print each token
print("Tokens:")
for token in tokens:
print(token)
Run
tokenization
D:\Work\NLP-lab\.venv\Scripts\python.exe D:\Work\NLP-lab\tokenization.py
Tokens:
Natural
Language
Processing
is
a
key
part
of
AI
and
data
science
.
Process finished with exit code 0
NLP-lab
tokenization.py
12:1
CRLF
UTF-8
4 spaces
Python 3.12 (NLP-lab)

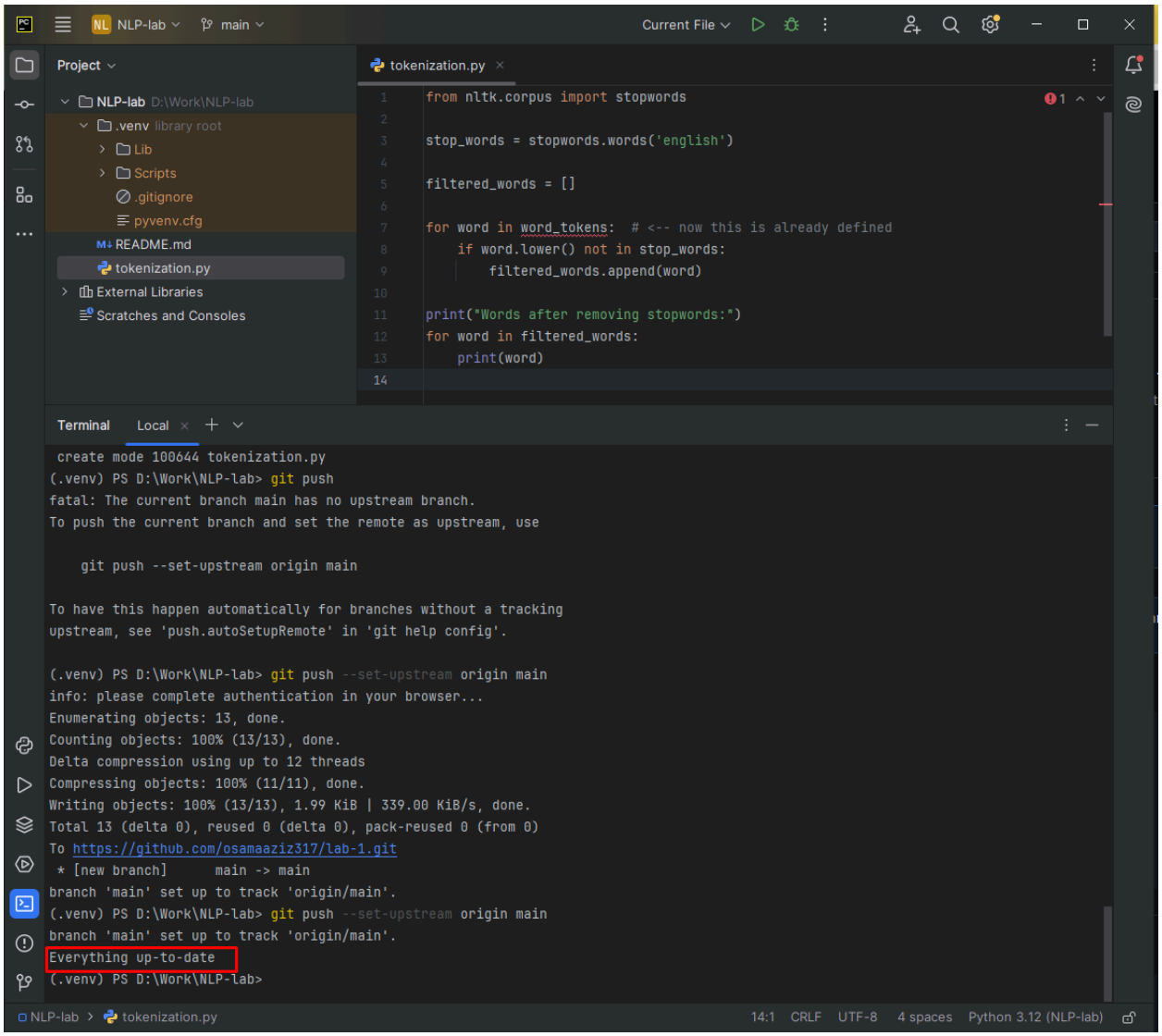

Dashboard
Type / to search
Top repositories
New
Find a repository...
osamaaziz317/insurgenbot
osamaaziz317/ai-form-filling-insurgen
Home
Ask Copilot
Make a Pong game
Create an issue for a bug
Get code feedback
Feed
Filter
Trending repositories · See more
google-gemini/gemini-cli
Star
An open-source AI agent that brings the power of Gemini directly into your terminal.
TypeScript
35.4k
coleam00/ottomator-agents
Star
All the open source AI Agents hosted on the oTTomator Live Agent Studio platform!
Python
2.5k
Latest changes
3 hours ago
Upcoming breaking change for GitHub Actions: Removal of...
Yesterday
Improved pull request files changed experience now in...
Yesterday
GitHub Actions fine-grain permissions are now generally...
Yesterday
GitHub Copilot coding agent is available for Copilot Pro users...
View changelog →
© 2025 GitHub, Inc.
Terms
Privacy
Security
Status
Docs
Contact
Manage cookies
Do not share my personal information

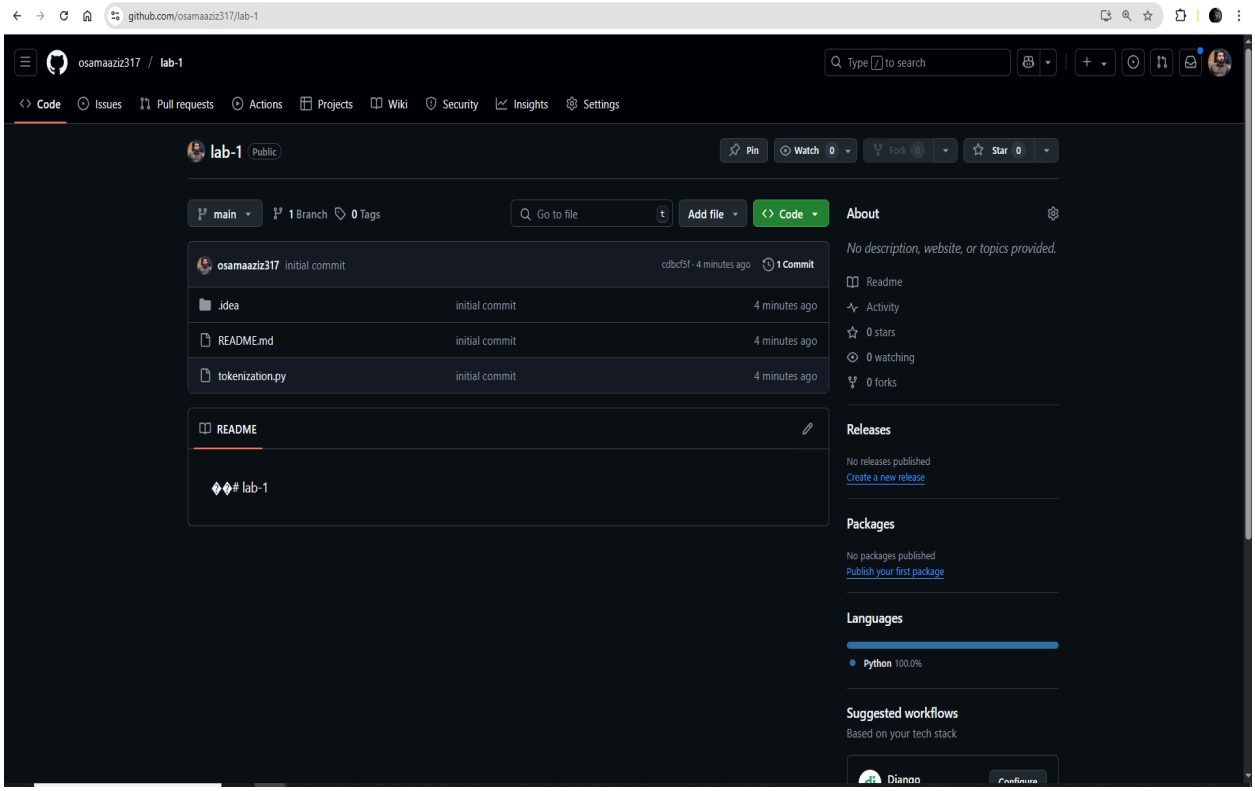

NLP-lab
Version control
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
filtered_words = []
for word in word_tokens: # <-- now this is already defined
if word.lower() not in stop_words:
filtered_words.append(word)
print("Words after removing stopwords:")
for word in filtered_words:
print(word)
Terminal
Local
Requirement already satisfied: regex>=2021.8.3 in d:\work\nlp-lab\.venv\lib\site-packages (from nltk==3.8.1) (2024.11.6)
Requirement already satisfied: tqdm in d:\work\nlp-lab\.venv\lib\site-packages (from nltk==3.8.1) (4.67.1)
Requirement already satisfied: colorama in d:\work\nlp-lab\.venv\lib\site-packages (from click->nltk==3.8.1) (0.4.6)
Downloading nltk-3.8.1-py3-none-any.whl (1.5 MB)
1.5/1.5 MB 3.6 MB/s eta 0:00:00
Installing collected packages: nltk
Successfully installed nltk-3.8.1
[notice] A new release of pip is available: 23.2.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\Work\NLP-lab>
(.venv) PS D:\Work\NLP-lab> git init
Initialized empty Git repository in D:/Work/NLP-lab/.git/
(.venv) PS D:\Work\NLP-lab>
NLP-lab > tokenization.py
14:1 CRLF UTF-8 4 spaces Python 3.12 (NLP-lab)


github.com/new
A repository contains all project files, including the revision history. Already have a project repository elsewhere?
Import a repository.
Required fields are marked with an asterisk (*).
Owner *
Repository name *
osamaaziz317
/
lab-1
lab-1 is available.
Great repository names are short and memorable. Need inspiration? How about symmetrical-parakeet ?
Description (optional)
Public
Anyone on the internet can see this repository. You choose who can commit.
Private
You choose who can see and commit to this repository.
Initialize this repository with:
Add a README file
This is where you can write a long description for your project. Learn more about READMEs.
Add .gitignore
.gitignore template: None
Choose which files not to track from a list of templates. Learn more about ignoring files.
Choose a license
License: None
A license tells others what they can and can't do with your code. Learn more about licenses.
You are creating a public repository in your personal account.
Create repository


NLP-lab
main
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
README.md
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
filtered_words = []
for word in word_tokens: # <-- now this is already defined
if word.lower() not in stop_words:
filtered_words.append(word)
print("Words after removing stopwords:")
for word in filtered_words:
print(word)
Terminal
Local
to set your account's default identity.
fatal: unable to auto-detect email address (got 'Osama Aziz@DESKTOP-55ERV6V.(none)')
(.venv) PS D:\Work\NLP-lab> git remote -v
origin https://github.com/osamaaziz317/lab-1.git (fetch)
origin https://github.com/osamaaziz317/lab-1.git (push)
(.venv) PS D:\Work\NLP-lab> git branch
(.venv) PS D:\Work\NLP-lab> git config --global osamaaziz317@gmail.com
error: invalid key: osamaaziz317@gmail.com
(.venv) PS D:\Work\NLP-lab> git config --global user.email "osamaaziz317@gmail.com"
(.venv) PS D:\Work\NLP-lab> git config --global user.name "osamaaziz317"
(.venv) PS D:\Work\NLP-lab> git add .
(.venv) PS D:\Work\NLP-lab> git commit -m "initial commit"
[main (root-commit) cdbcf5f] initial commit
9 files changed, 68 insertions(+)
create mode 100644 .idea/.gitignore
create mode 100644 .idea/NLP-lab.iml
create mode 100644 .idea/inspectionProfiles/Project_Default.xml
create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 100644 README.md
create mode 100644 tokenization.py
NLP-lab > tokenization.py
14:1 CRLF UTF-8 4 spaces Python 3.12 (NLP-lab)


NLP-lab
main
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
README.md
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
filtered_words = []
for word in word_tokens: # <-- now this is already defined
if word.lower() not in stop_words:
filtered_words.append(word)
print("Words after removing stopwords:")
for word in filtered_words:
print(word)
Terminal
Local
(.venv) PS D:\Work\NLP-lab> git config --global user.name "osamaaziz317"
(.venv) PS D:\Work\NLP-lab> git add .
(.venv) PS D:\Work\NLP-lab> git commit -m "initial commit"
[main (root-commit) cdbcf5f] initial commit
9 files changed, 68 insertions(+)
create mode 100644 .idea/.gitignore
create mode 100644 .idea/NLP-lab.iml
create mode 100644 .idea/inspectionProfiles/Project_Default.xml
create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 100644 README.md
create mode 100644 tokenization.py
(.venv) PS D:\Work\NLP-lab> git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use
git push --set-upstream origin main
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
(.venv) PS D:\Work\NLP-lab> git push --set-upstream origin main
Connect to GitHub
GitHub
Sign in
Browser/Device
Token
Sign in with your browser
Sign in with a code
Don't have an account? Sign up


NLP-lab
main
Current File
Project
NLP-lab D:\Work\NLP-lab
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
README.md
tokenization.py
External Libraries
Scratches and Consoles
tokenization.py
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
filtered_words = []
for word in word_tokens: # <-- now this is already defined
if word.lower() not in stop_words:
filtered_words.append(word)
print("Words after removing stopwords:")
for word in filtered_words:
print(word)
Terminal
Local
create mode 100644 tokenization.py
(.venv) PS D:\Work\NLP-lab> git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use
git push --set-upstream origin main
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
(.venv) PS D:\Work\NLP-lab> git push --set-upstream origin main
info: please complete authentication in your browser...
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (13/13), 1.99 KiB | 339.00 KiB/s, done.
Total 13 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/osamaaziz317/lab-1.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
(.venv) PS D:\Work\NLP-lab> git push --set-upstream origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
(.venv) PS D:\Work\NLP-lab>
NLP-lab > tokenization.py
14:1 CRLF UTF-8 4 spaces Python 3.12 (NLP-lab)


github.com/osamaaziz317/lab-1
osamaaziz317 / lab-1
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
lab-1
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
osamaaziz317 initial commit
cdbcf5f · 4 minutes ago
1 Commit
.idea
initial commit
4 minutes ago
README.md
initial commit
4 minutes ago
tokenization.py
initial commit
4 minutes ago
README
lab-1
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Languages
Python 100.0%
Suggested workflows
Based on your tech stack
Django
Configure